UNIVERSITI TEKNIKAL MALAYSIA MELAKA

FACULTY OF ARTIFICIAL INTELLIGENCE AND CYBER SECURITY

W O R K S H O P 1

R E P O R T

Name : SOONG JIEN YOONG

Matric Number : B032410365

Program : 2 BAXI S1G2

Project Title : LIBRARY MANAGEMENT SYSTEM

Supervisor Name : TS. DR. NORFADZLIA BINTI MOHD YUSOF

Evaluator Name : PM TS. DR. HALIZAH BINTI BASIRON

EXECUTIVE SUMMARY

The Library Management System is developed to digitize and streamline the process of managing library resources with greater efficiency, replacing the traditional manual system that often causes human errors and time-consuming tasks. The current manual system suffers from inefficiency and inaccuracy, as staff members face difficulties in tracking book availability and borrower details, while students experience challenges in searching for or borrowing specific books. Therefore, implementing an automated system that can store, update, and retrieve library records more quickly and accurately will significantly improve the overall efficiency of library management. The main objectives of this project are to design and develop a digital database system to manage information on books, students, and staff; to minimize human errors through an automated borrowing and returning process; and to provide users with fast and convenient access to available books. The scope of this system covers six key modules: Login & Authentication Module, Data Management Module, Book Loan Management Module, Advanced Searching Module, User Review and Rating Module, and Report Generation Module. This project is significant as it enhances the accuracy, accessibility, and efficiency of the library system. By implementing this digital solution, the library can provide faster and more reliable services to both staff and students, fostering a more organized and effective learning environment.

TABLE OF CONTENTS

PAGE

LIST OF TABLES

PAGE

LIST OF FIGURES

PAGE

CHAPTER 1: INTRODUCTION

1.1 Introduction

A library plays an important role in providing academic resources for study and research. A Library Management System is a digital and automated database designed to replace the existing manual process, making overall management more efficient, organized, and systematic. The system allows administrators and staff to record and manage information related to staff, students, books, borrowing, and returning activities with greater accuracy and speed.

1.2 Problem Statement

The current manual library system faces several challenges such as difficulties in sorting and tracking data, time-consuming operations, and higher labour costs. It also increases the chances of human errors, missing records, and data loss due to a lack of data integrity, which affects the overall reliability and efficiency of library operations. Therefore, developing a digital Library Management System can help overcome these issues by automating key processes, improving data accuracy, and enhancing the overall management of library resources for a more effective and organized library environment.

1.3 Objective (s) of the project

This project embarks on the following objectives:

1. To implement six modular functionalities with CRUDS (Create, Read, Update, Delete, Search) database management for the Library Management System:
    i. User Management – Handles the user login system and manages access permissions.
    ii. Data Management – Adds, deletes, and updates all system data efficiently.
    iii. Books Loan Management – Manages book borrowing and returning processes, including fine tracking.

iv. User Review and Rating Module – Displays book reviews and ratings for users to view

v. Advanced Searching – Provides a search engine to find books using keywords or filters.

vi. Report Generation – Generates reports such as book popularity and user loan history.

2. To perform complex calculations related to book borrowing durations, fines for late-returned books, borrowing limits for different user types (staff and students), and the average rating of reviewed books through the advanced searching feature.

3. To develop a report generation module that automatically analyses borrowing records, book popularity, and user activities to provide detailed reports for administrators and staff.

1.4 Scope

1.4.1 Modules to be developed

a) Manages admin and staff accounts, roles, and access permissions.

b) Stores and updates information about books, staff, and students efficiently.

c) Handles the borrowing and returning of books with automatic fine calculation.

d) Allows users to give feedback and rate books they have borrowed.

e) Helps users find books easily using keywords, titles, or authors.

f) Generates reports on book popularity and user borrowing history.

1.4.2 Target User

a) Administrator / Manager

- Has full access through the Login & Authentication Module to manage user accounts and permissions.
- Uses the Data Management Module to manage and update staff, student, and book records.
- Oversees the Books Loan Management Module to monitor borrowing and returning activities.
- Can access the Report Generation Module to view summary reports such as most borrowed books, and fine records for decision-making.

b) Library Staff

- Logs in through the Login & Authentication Module with assigned staff access rights.
- Uses the Data Management Module to update book details and student borrowing records.
- Manages borrowing and returning processes using the Books Loan Management Module, including fine calculation.
- Can use the Advanced Searching Module to find specific books or records efficiently.
- Has limited access to the Report Generation Module for viewing daily or weekly summaries.

c) Students / Borrowers

- Can use the Advanced Searching Module to find books based on title, author, or category.

- Access the User Review and Rating Module to read or submit book reviews and ratings.

1.5 Project Significance

- To improve efficiency in managing library operations by automating manual processes.

- To reduce human errors and prevent data loss through a reliable digital database.

- To save time for both staff and students when borrowing, returning, and tracking books.

- To enhance the accuracy and organization of records and reports. • To provide a more user-friendly and accessible system for staff and students.

- To support a better academic environment through improved access to learning resources.

1.6 Gantt Chart of Project Activities

Table 1.6.1: Gantt Chart of Project Activities

No	Task	WEEK														
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	Discussion / Verification of title and synopsis															
2	Proposal preparation															
3	Discussion with supervisor on analysis of the problem.															
4	Project Implementation															
5	Final Presentation & Submission of Final Report															

CHAPTER 2: ANALYSIS OF PROBLEM

2.1 Problem Decomposition Description

Table 2.1.1: Problem Decomposition Description

No	Identified Problem	Proposed Solution (New System)
1.	User management problem	Implement secure login and role-based access control for administrators and staff.
2.	Difficult in data management	Develop a centralized digital database that allows efficient data storage, updating, and retrieval.
3.	Record data of borrowing and returning books manually is time-consuming and prone to an errors	Automate the borrowing and returning process with real-time updates and automatic fine calculation.
4.	No system for collecting user feedback or book popularity	Introduce a review and rating feature for users to provide feedback and help improve library collections.
5.	Students and staff face difficulties finding books using the manual system	Implement a search function that allows users to find books easily using keywords, titles, or authors.
6.	Manual report preparation is slow and lacks accuracy	Automate report generation to produce accurate and detailed reports on book usage and user activities.

2.2 Structured Chart

Figure 2.2.1: Structured Chart

CHAPTER 3: DESIGN

3.1 Business Rules

Table 3.1.1: Business rule

Module	Allowed operation	Workflow	Constraints (System Limits)	Validation Rules
Login and Authentication Module	Admin and Staff can log in. Role-based access after successful login.	User enters username and password. System checks credentials. If correct, access granted based on role. If incorrect, error message displayed.	Only Admin and Staff accounts can log in. Students cannot access the login module. Deleted accounts are not allowed to log in.	Username and password fields cannot be empty. Password must match stored encrypted password.

Data management module (Admin, Staff, Book, Student)	Admin can add /update /delete staff and admin records. Staff can add/update books and student records. Student data is created only during first borrowing.	User selects Add, Edit, Delete data System checks user role and grants permission accordingly. Admin/Staff fills in the form. System validates and saves into the database. Records are updated or removed based on the action	Only Admin can manage staff/admin records. Students cannot modify their own data. Admin ID, Book ID, Staff ID, Student ID must be unique.	Required fields such ID, Name, Email must not be empty. Email format must be valid
Book Loan Management Module	Only staff handle the borrowing and returning of books. System calculates fines according to the due dates	Staff select student & book. System checks the availability of the book Record is saved with borrow date.	Only students can borrow books with the help of staff. Staff/admin cannot borrow from the system. Book must be “available” to be borrowed.	Borrow date and due date must meet a valid year. A late return fine of RM1 per day will be added to the fines for each day, the book exceeds the due date.

User Review and Rating Module	Students (through staff assistance) can submit reviews and ratings for books. Anyone can view reviews (no login required).	After student return the books, he can give rating and feedback to Staff. Staff updates the loan and enters books review into system. System saves and displays reviews under the book.	Only users who have borrowed the book can leave a review. One user may only review a book once.	Rating must be between 1 and 5.
Advanced Searching Module	All users can search for books.	User enters book name. System searches in book database. Results show Id, Title, Author, Publisher, date of Publish, Status, and Category. If no result → display “No books found”.	Deleted books must not appear.	Search field cannot be empty.

Report generation module	Admin can view on the full reports. Staff can only view loans and review reports.	User selects report type System displays results according to the needs	Only admins have full access on the whole report. Reports cannot be edited directly in the system.	System must validate that selected report type exists.

3.2 Flowchart

3.2.1 Main

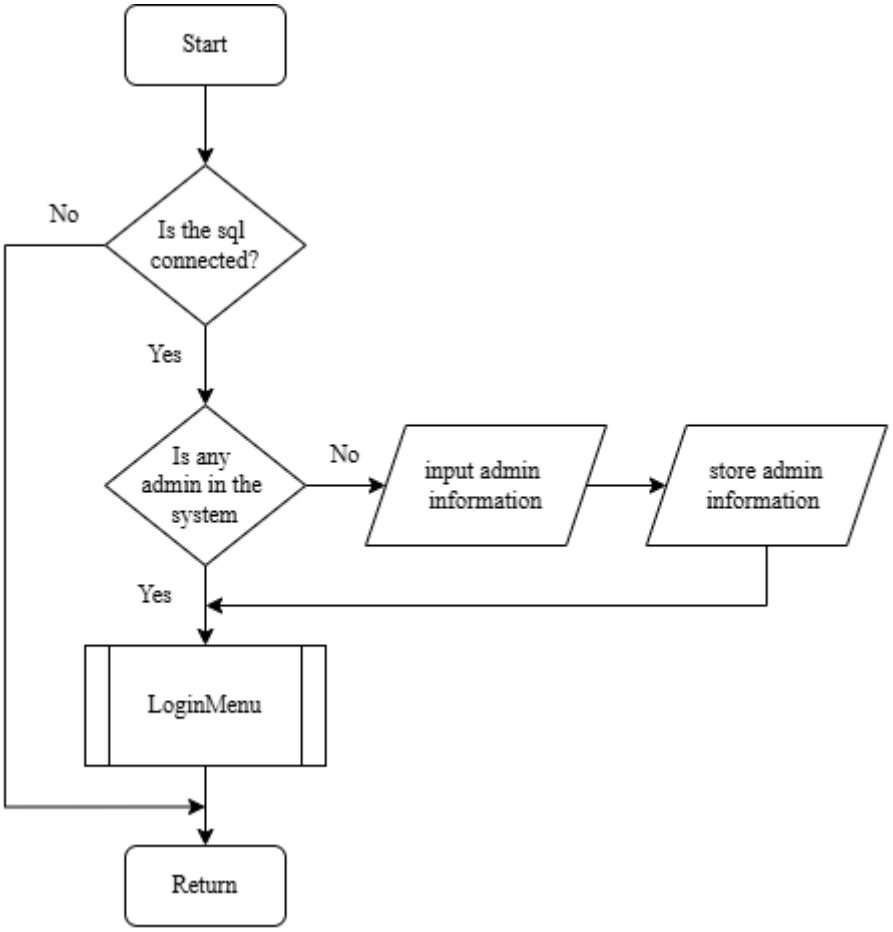

Figure 3.2.1: Main Menu flowchart

3.2.2 Login Module

3.2.2.1 Login Menu

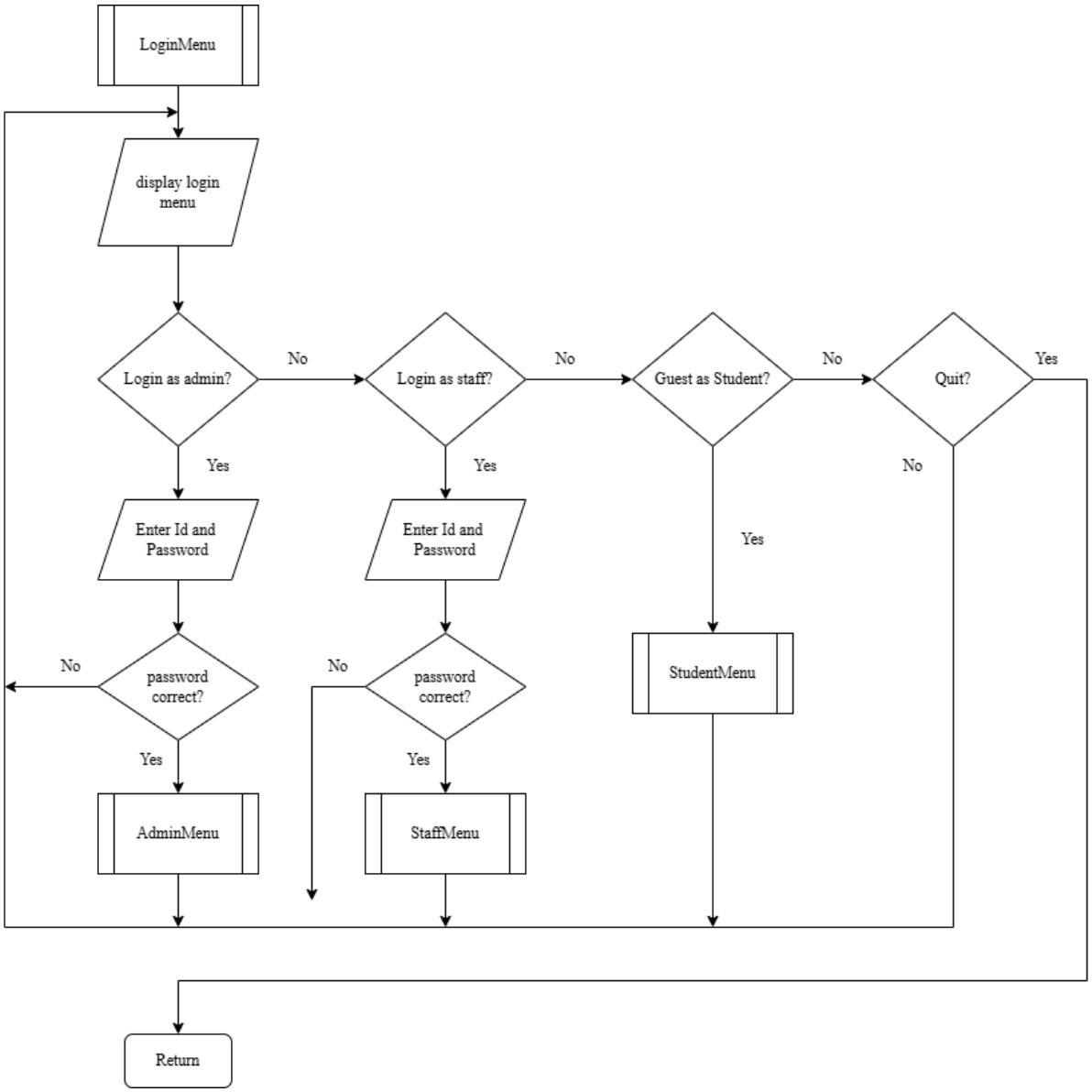

Figure 3.2.2: Login Menu flowchart

3.2.2.2 Admin Menu

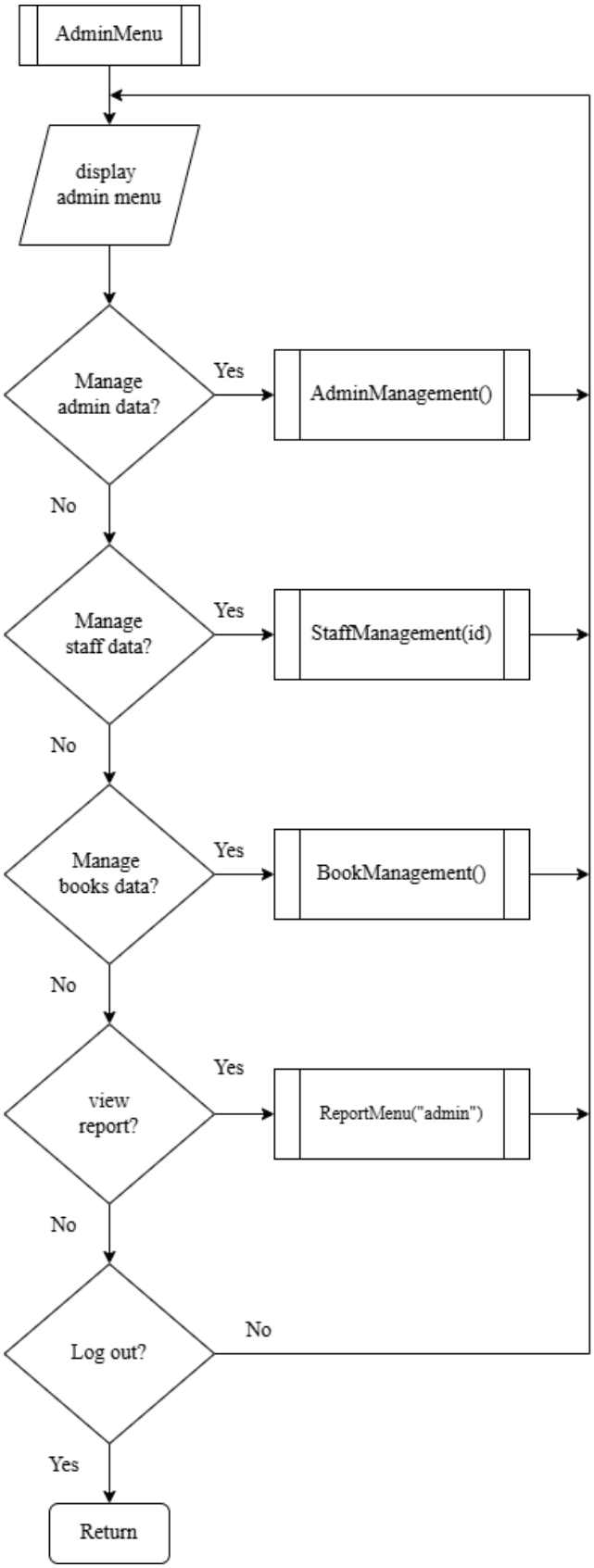

Figure 3.2.3: Admin Menu flowchart

3.2.2.3 Staff Menu

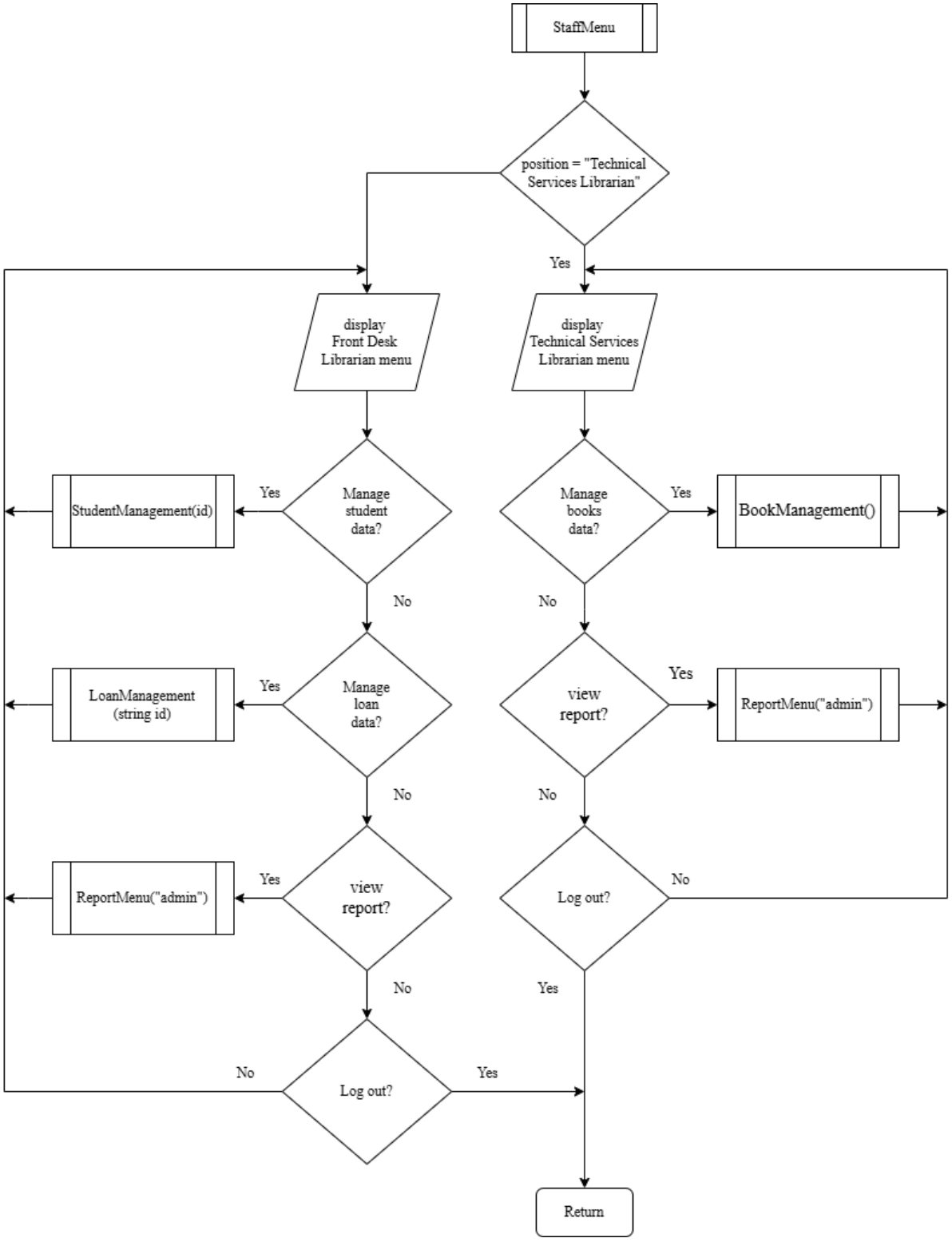

Figure 3.2.4: Staff Menu flowchart

3.2.2.4 Student Menu

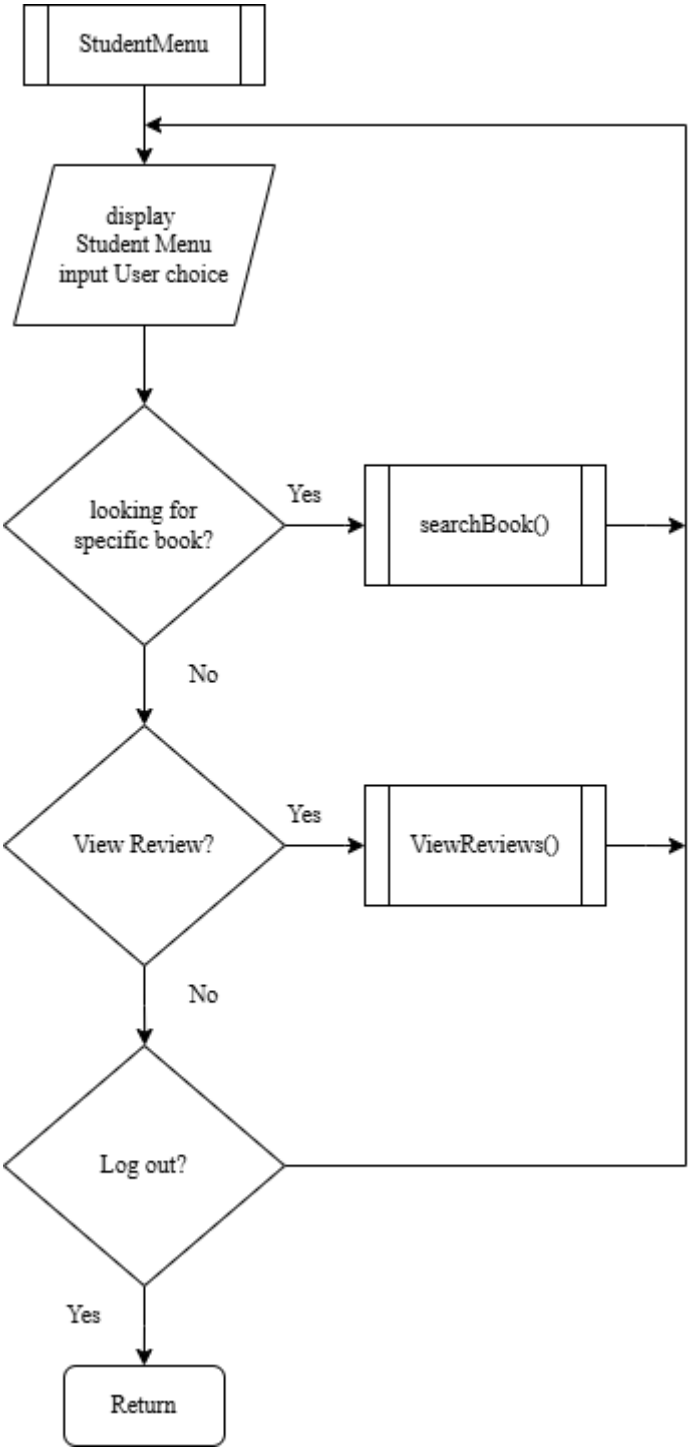

Figure 3.2.5: Student Menu flowchart

3.2.3 Data Management Module

3.2.3.1 Data Management Menu (Admin, staff, student, book)

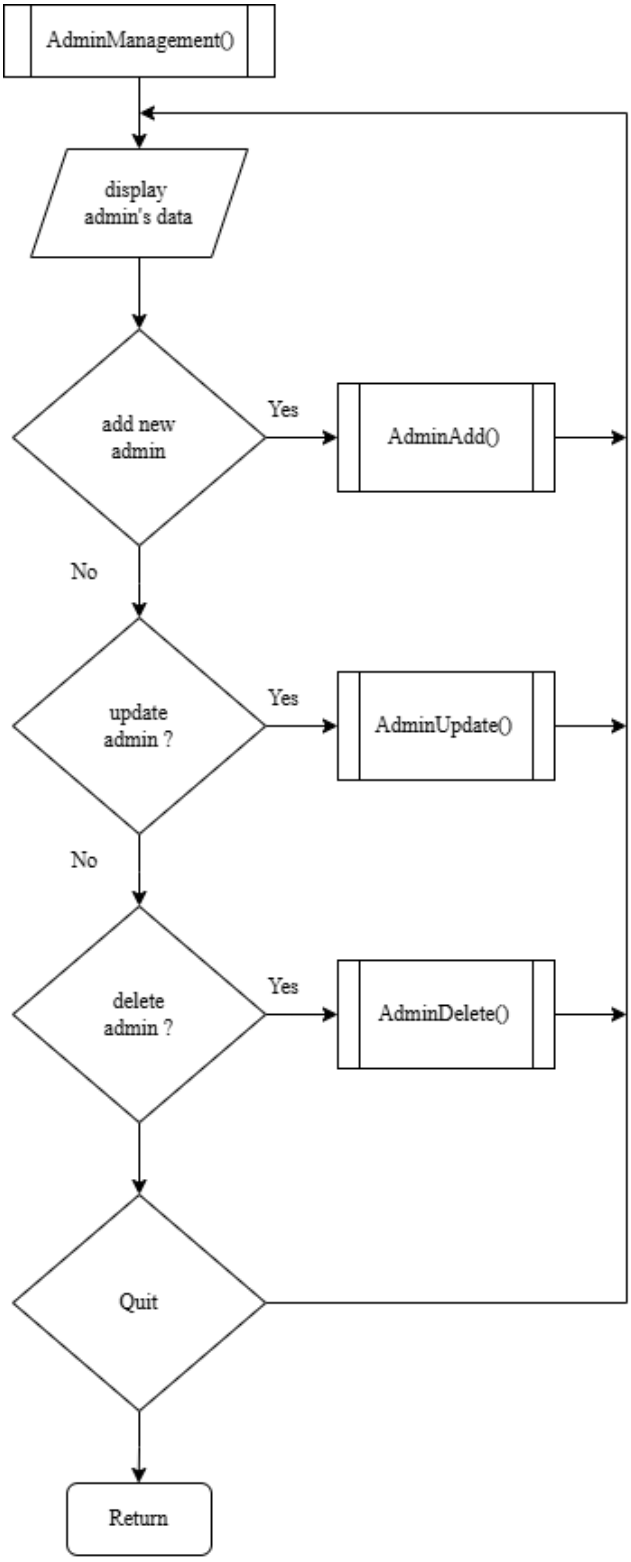

Figure 3.2.6: Admin Management Menu flowchart

3.2.3.2 Add Operation (Admin, staff, student, book)

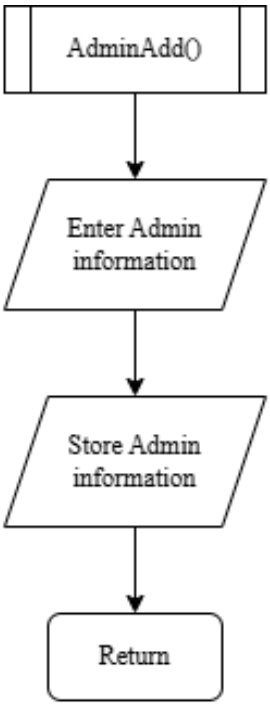

Figure 3.2.7: Data Add flowchart

3.2.3.3 Update Operation (Admin, staff, student, book)

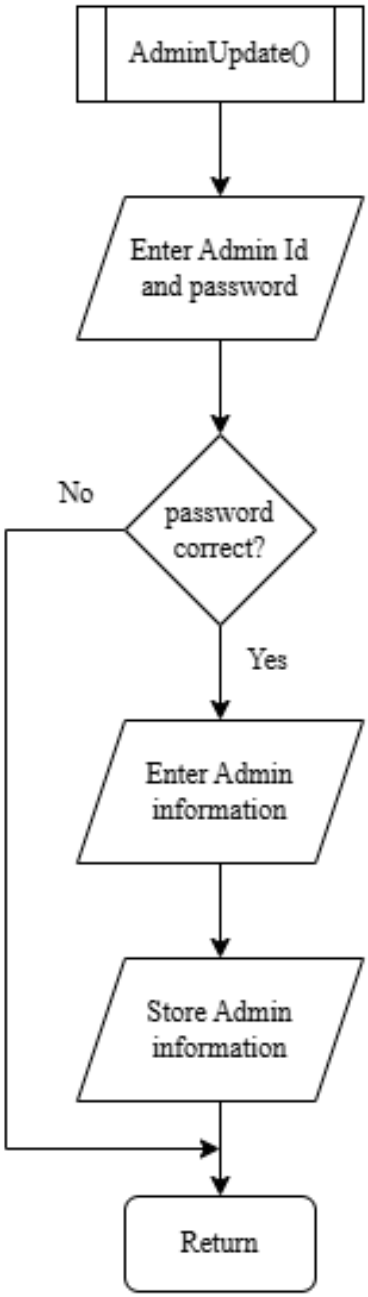

Figure 3.2.8: Data Update flowchart

3.2.3.4 Delete Operation (Admin, staff, student, book)

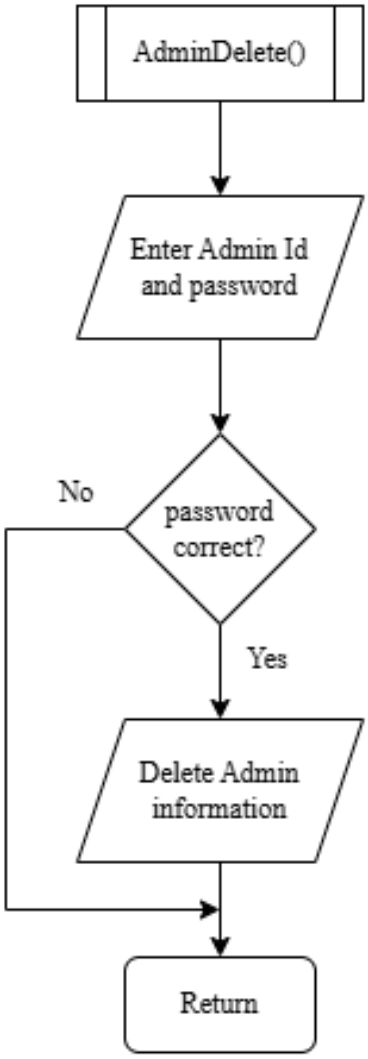

Figure 3.2.9: Data Delete flowchart

3.2.4 Loan Management Module

3.2.4.1 Loan Menu

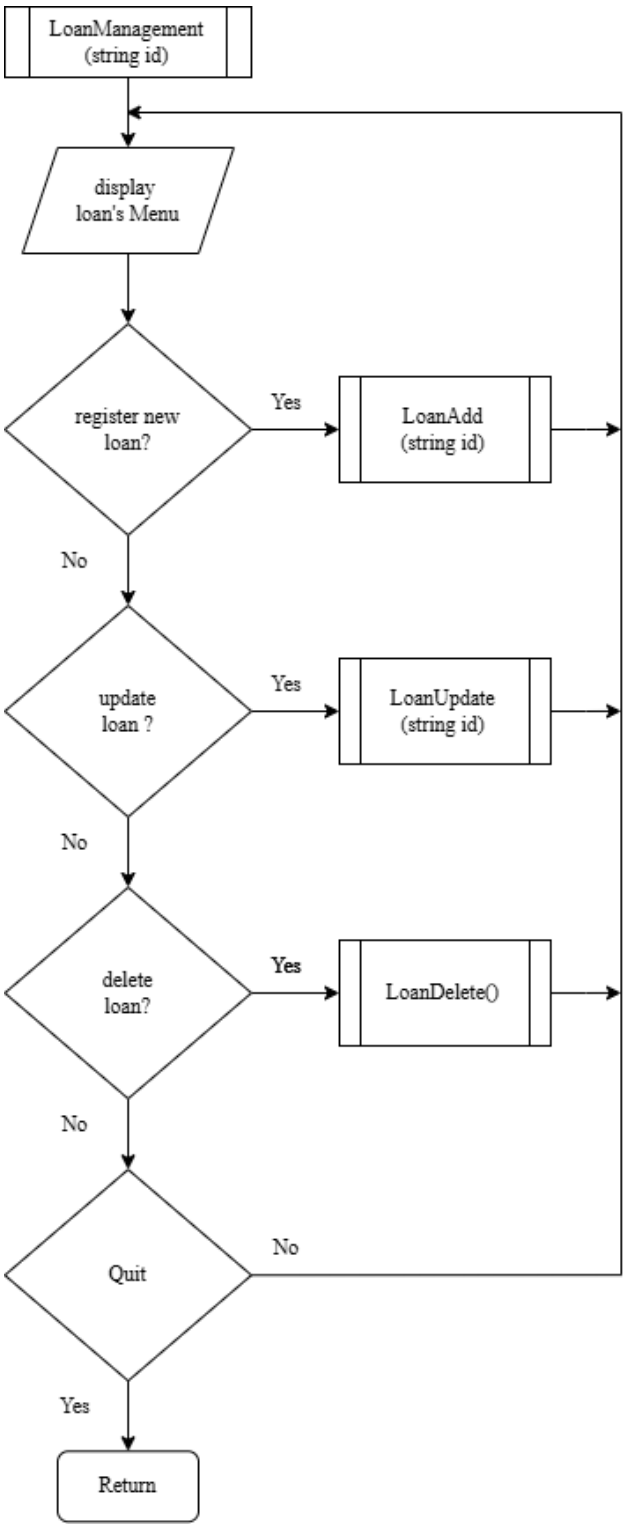

Figure 3.2.10: Staff Data flowchart

3.2.4.2 Loan Add Operation

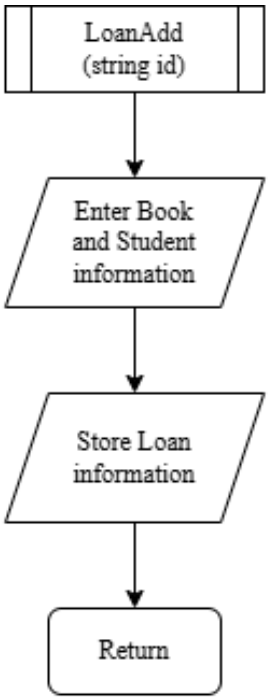

Figure 3.2.11: Loan Add flowchart

3.2.4.3 Loan Update Operation

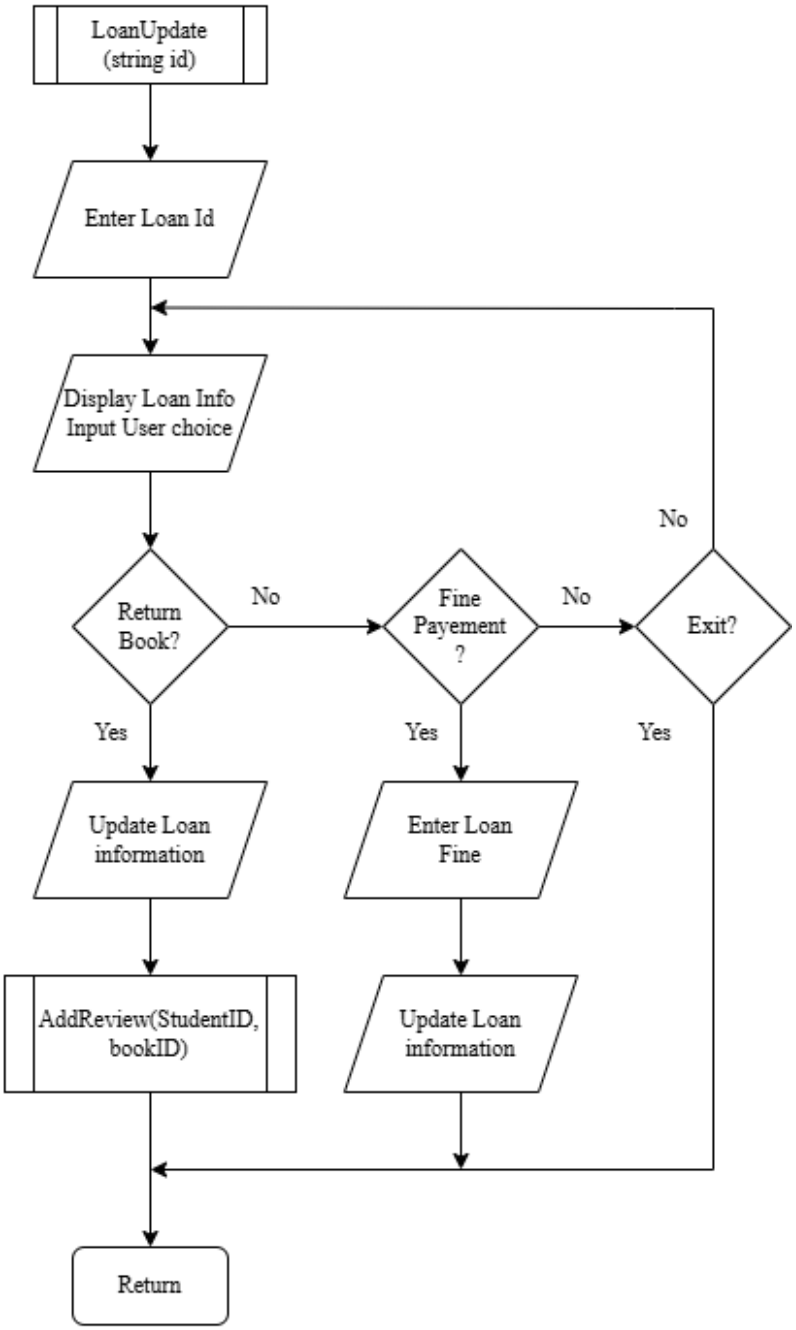

Figure 3.2.12: Loan update flowchart

3.2.4.4 Loan Delete Operation

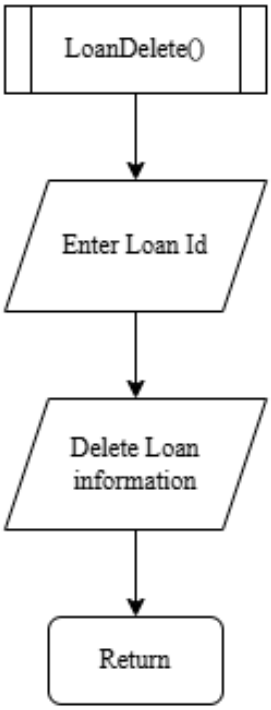

Figure 3.2.13: Loan Delete flowchart

3.2.5 Review and Rating Module

3.2.5.1 Add Review Operation

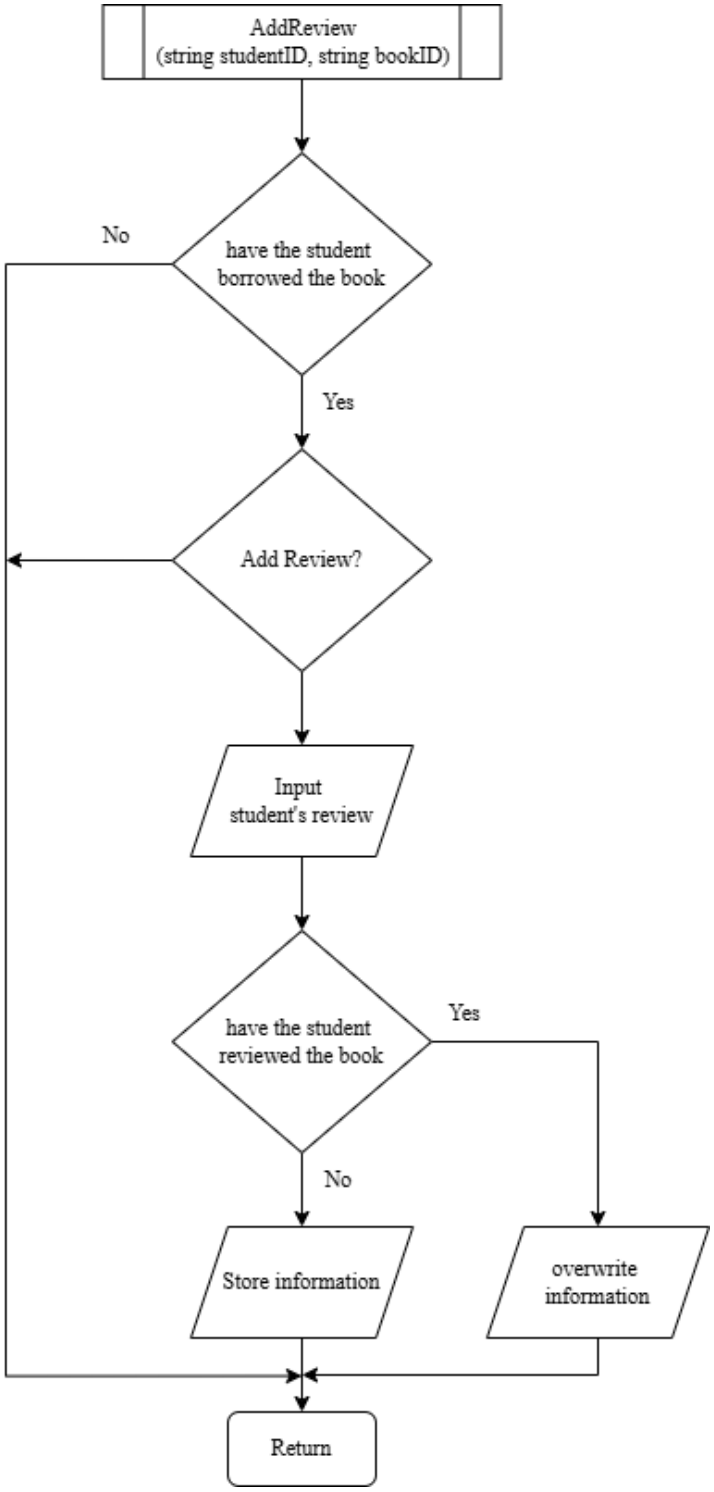

Figure 3.2.14: Add Review flowchart

3.2.5.2 View Review Function

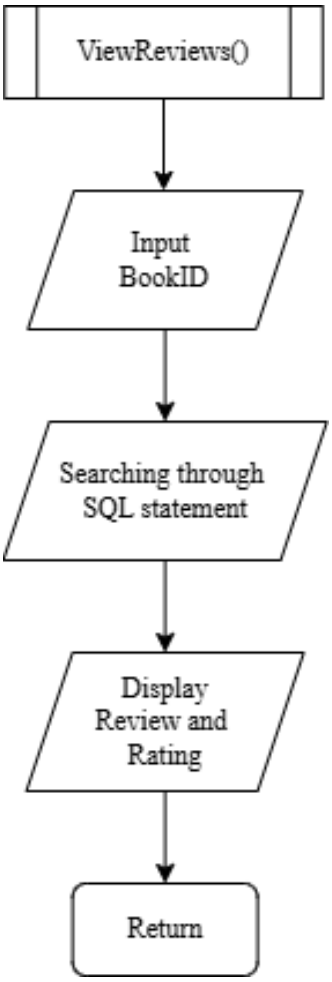

Figure 3.2.15: View Review flowchart

3.2.6 Advanced Searching Module

Figure 3.2.16: Advanced Search flowchart

3.2.7 Report Module

3.2.7.1 Report Menu

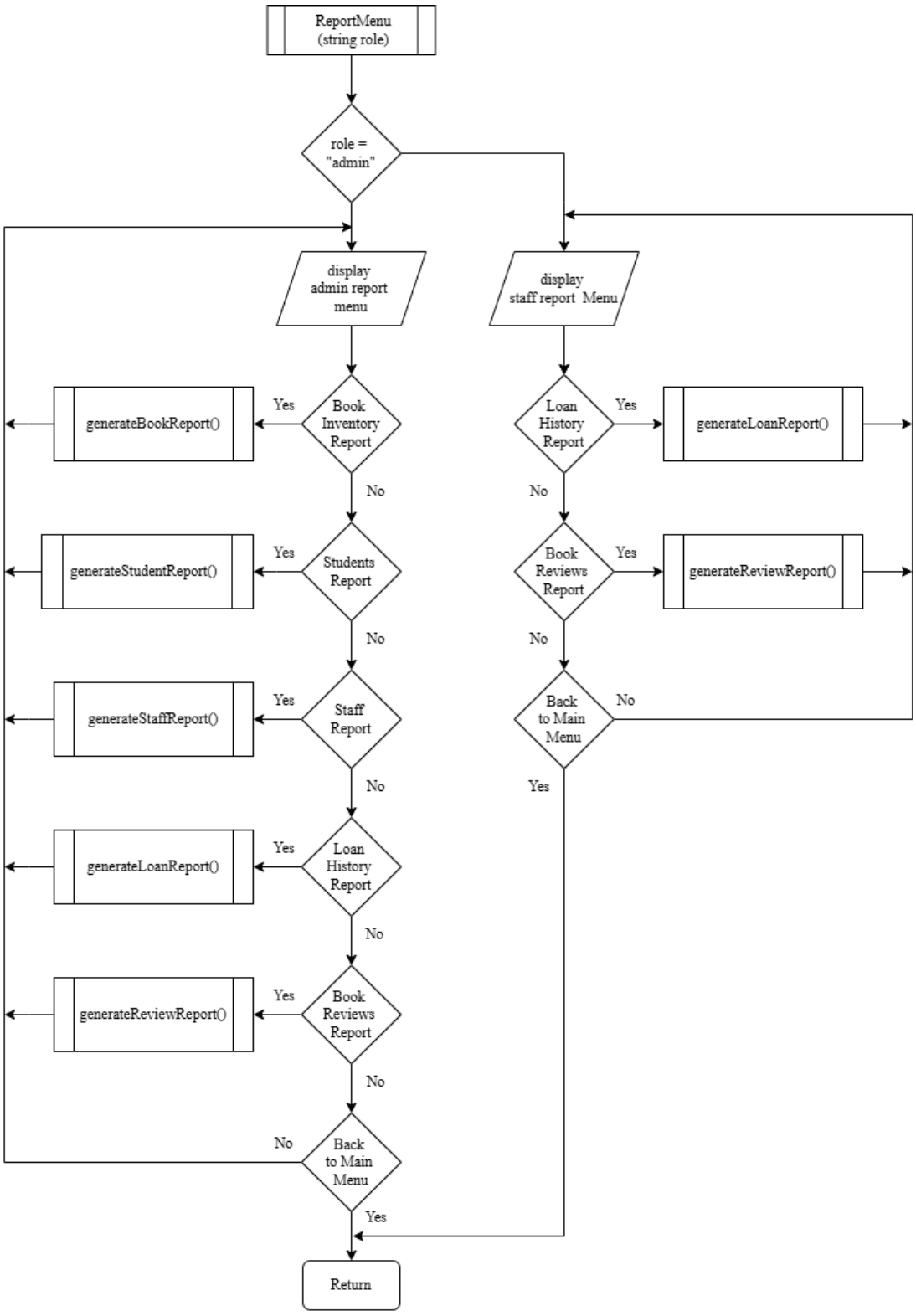

Figure 3.2.17: Report Menu flowchart

3.2.7.2 Simple Report Function (Book, Staff, Loan)

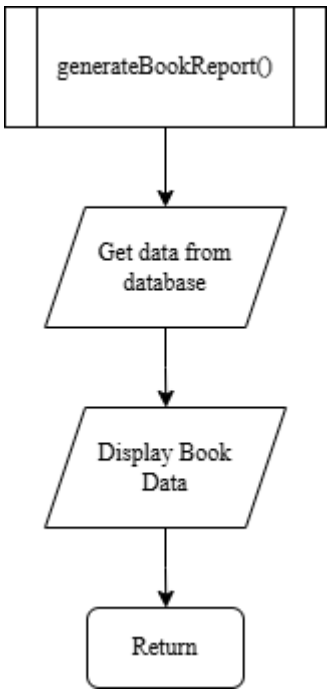

Figure 3.2.18: Book Report flowchart

3.2.7.3 Complex Report Function (Student, Review)

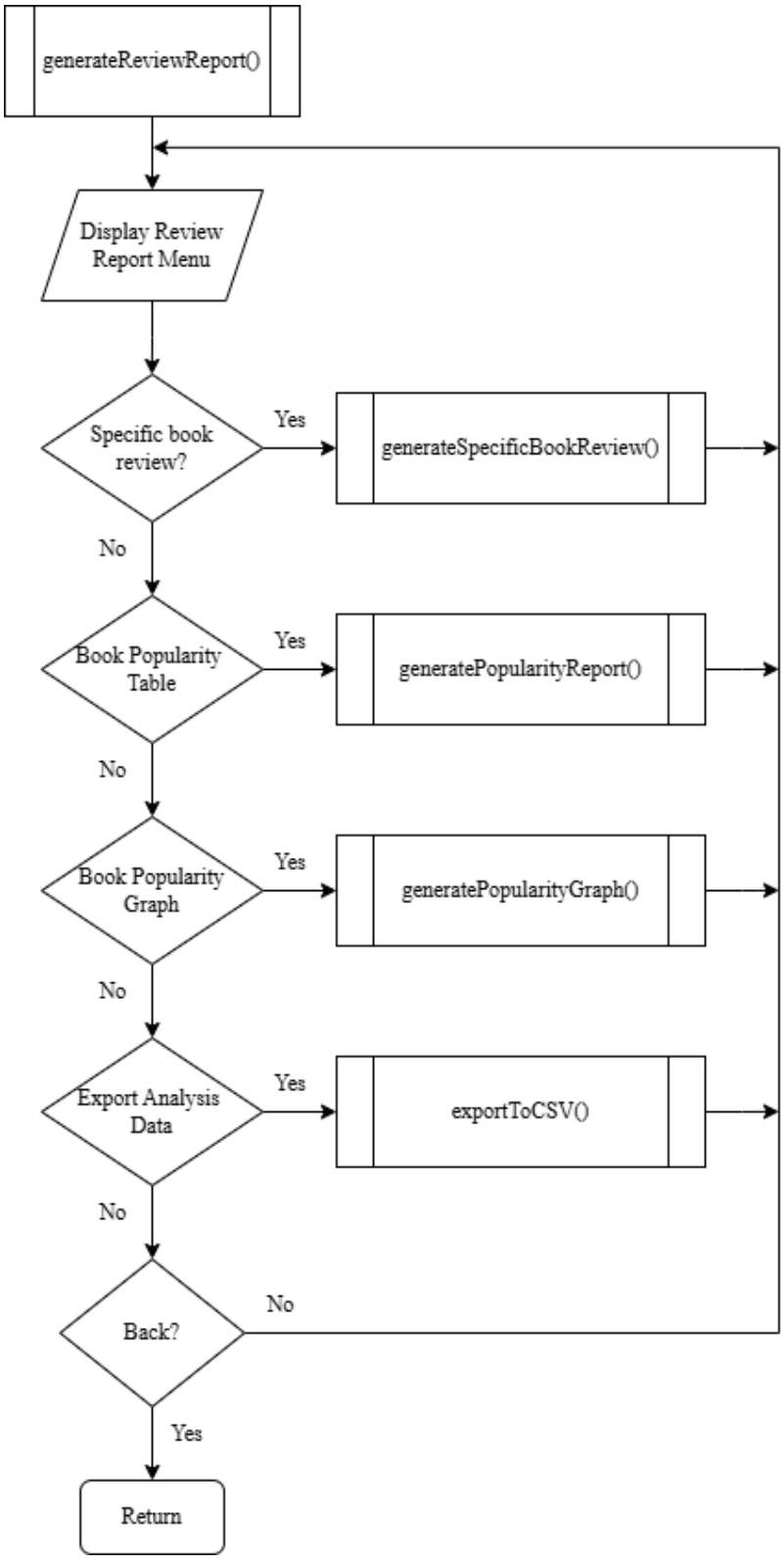

Figure 3.2.19: Review Report Menu flowchart

3.2.7.4 Specific Book Review Report

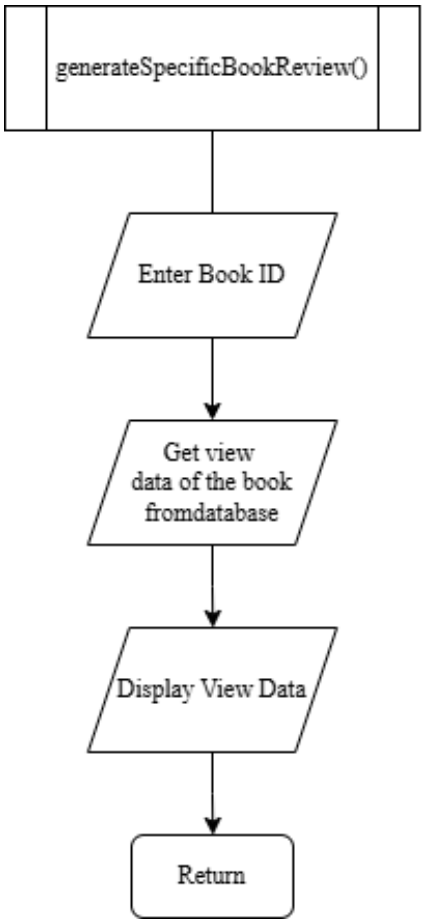

Figure 3.2.20: Specific Book Review Report flowchart

3.2.7.5 Book Popularity Report

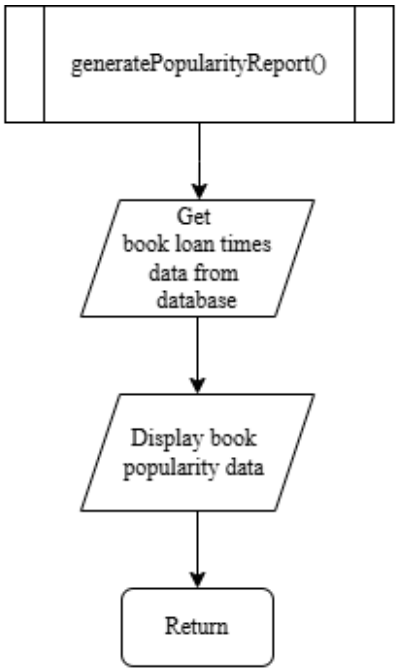

Figure 3.2.21: Book Popularity Report flowchart

3.2.7.6 Book Popularity Graph Report

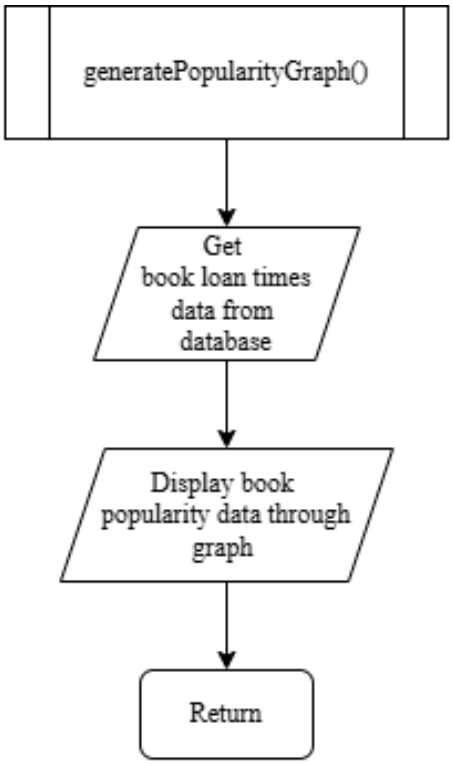

Figure 3.2.22: Book Popularity Graph Report flowchart

3.2.7.7 Export Book Popularity Report CSV

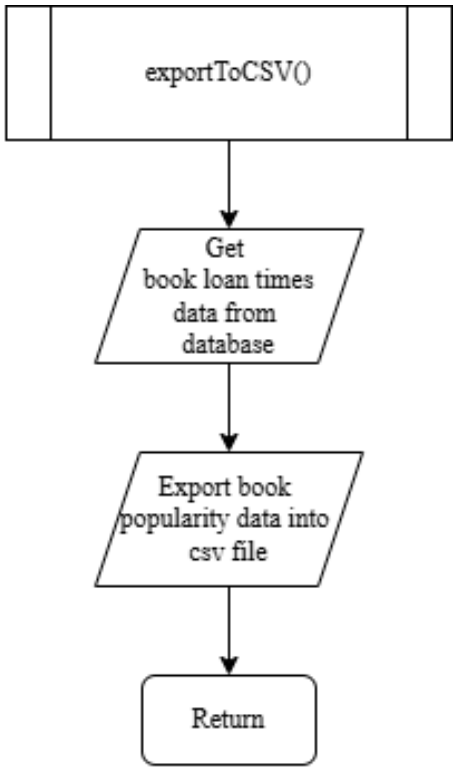

Figure 3.2.23: Export Book Popularity Report CSV flowchart

3.3 ERD

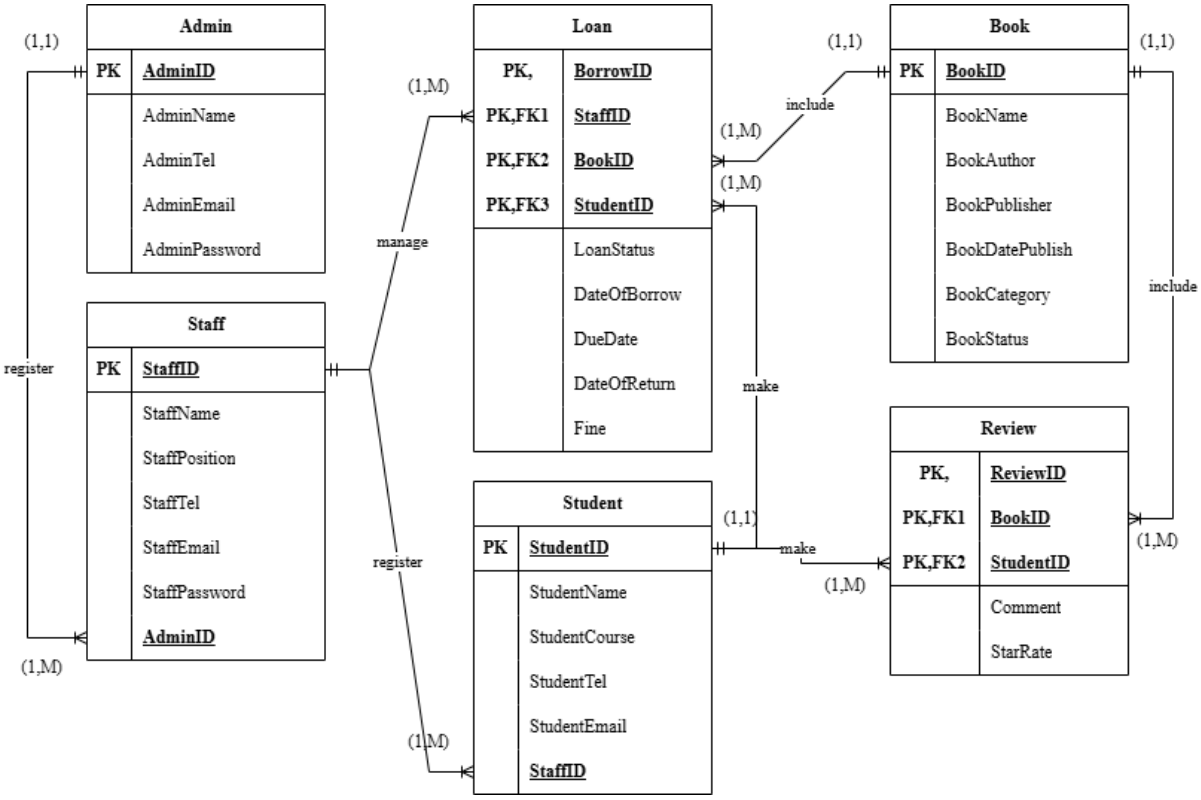

Figure 3.3.1: Data Model Entity Relationships Diagram

3.4 Data Normalization

3.4.1 Admin Register Staff Data

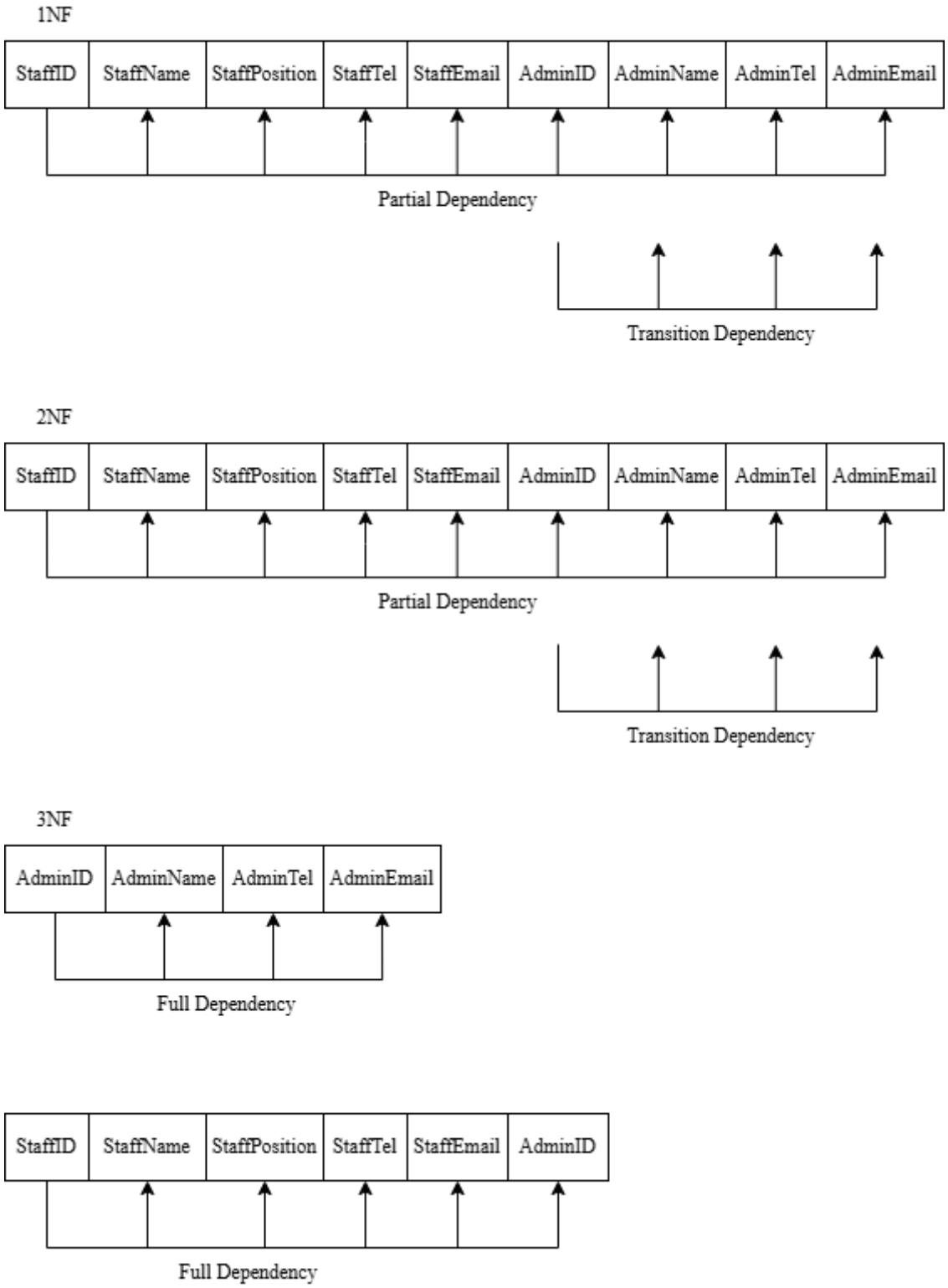

Figure 3.4.1: Data Normalisation of Admin Register Staff Data

3.4.2 Staff Register Student Data

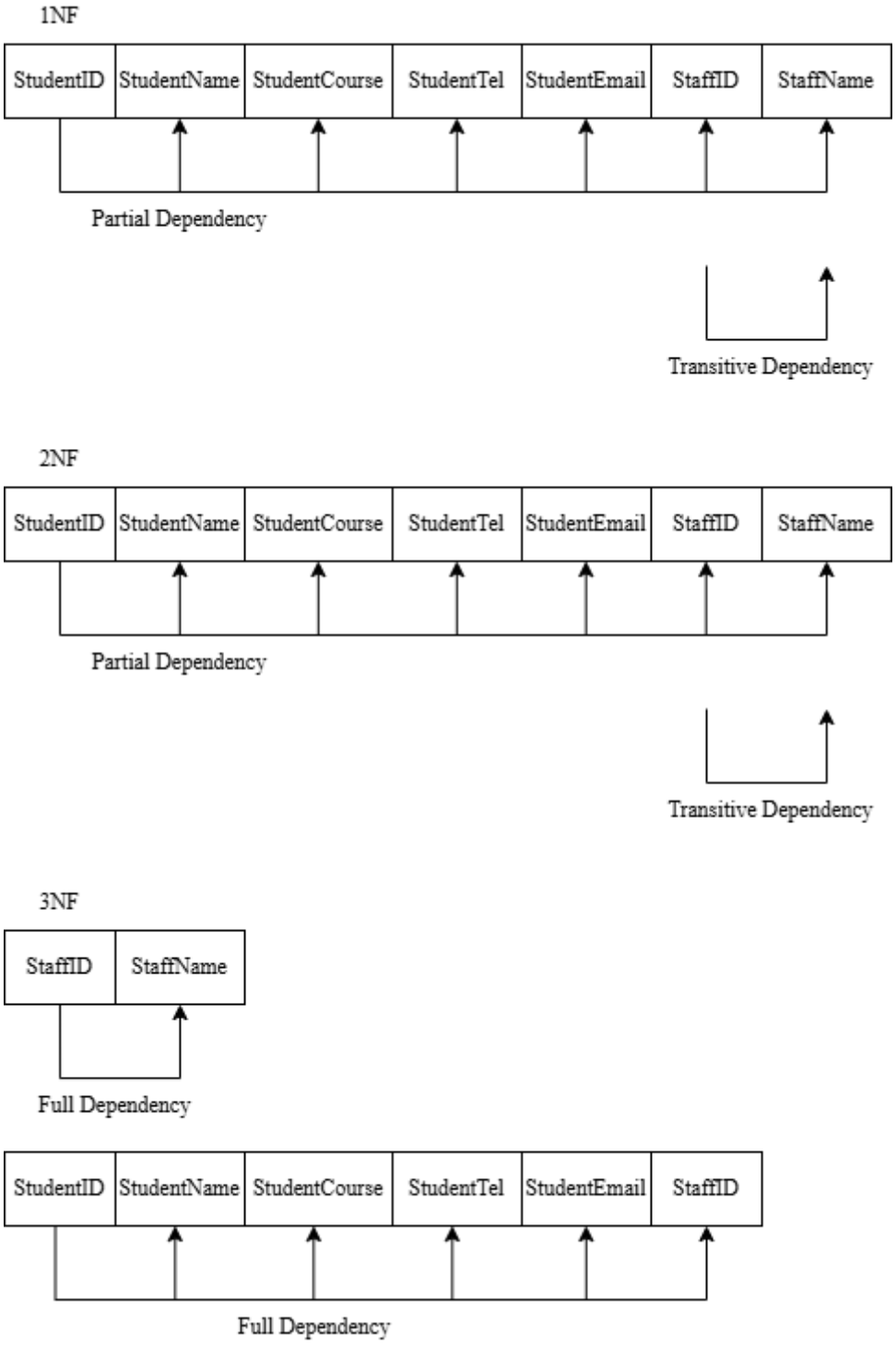

Figure 3.4.2: Data Normalisation of Staff Register Student Data

3.4.3 Books Review Data

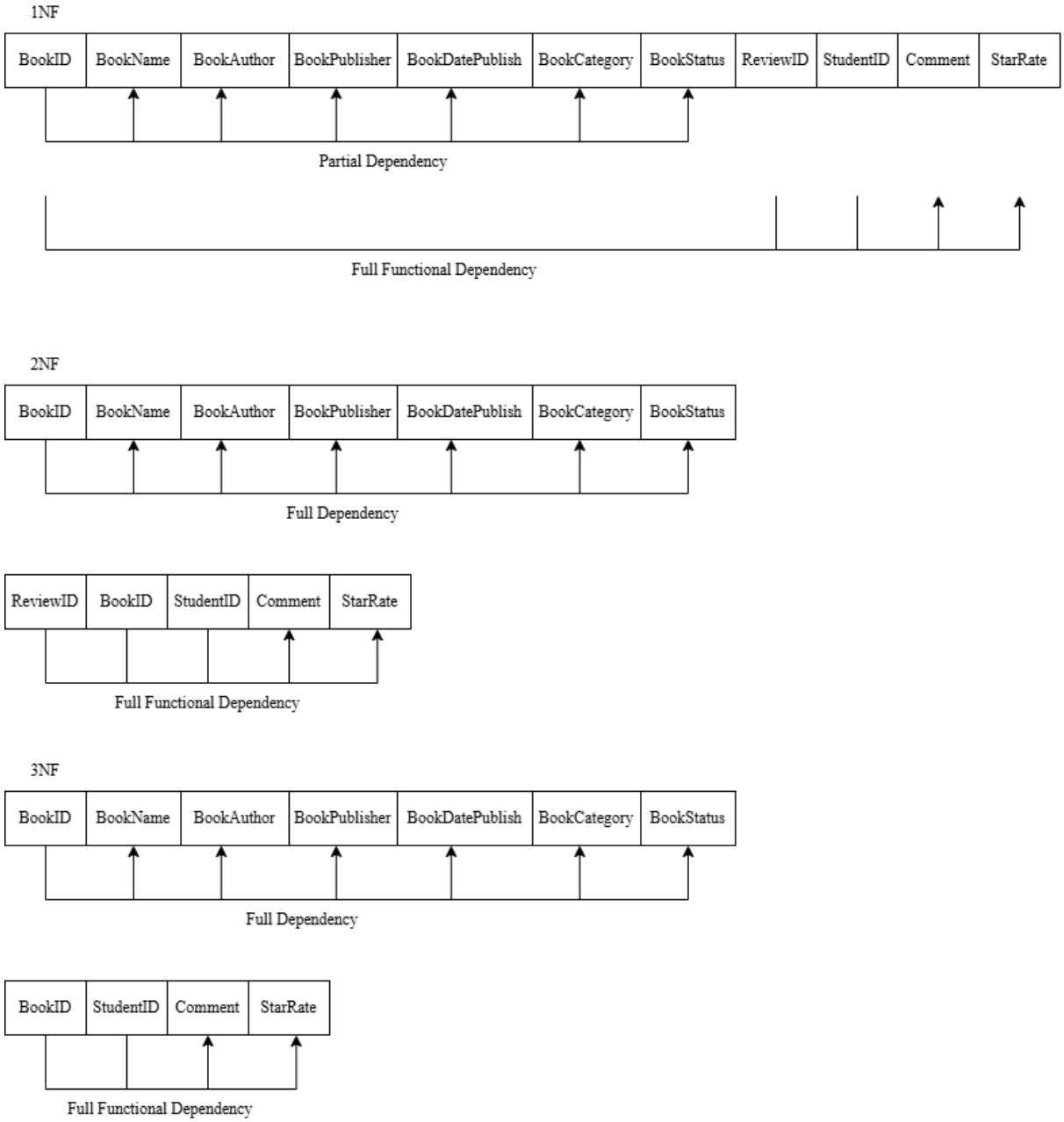

Figure 3.4.3: Data Normalisation of Books Review Data

3.4.4 Loan Data

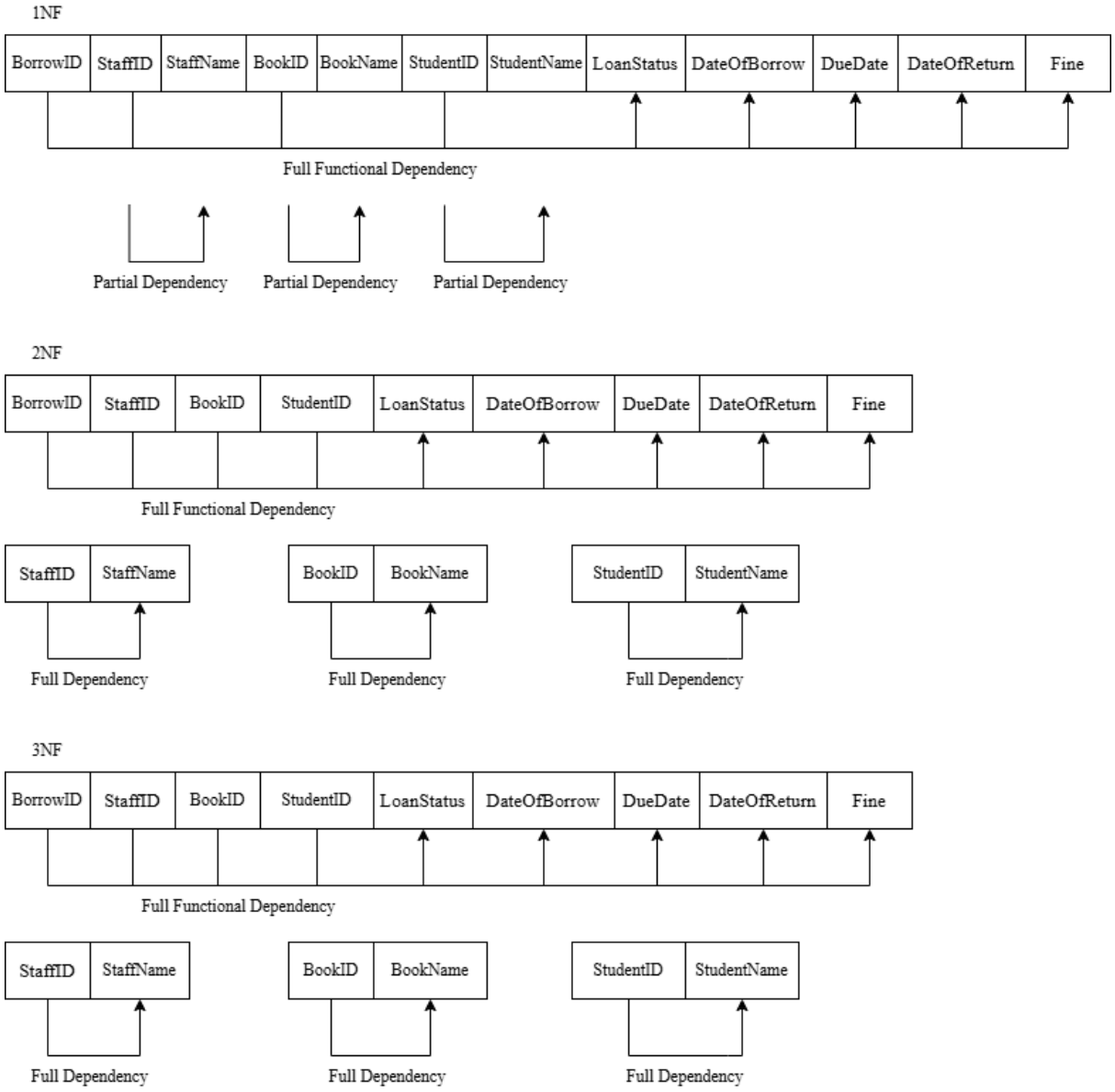

Figure 3.4.4: Data Normalisation of Loan Data

3.5 Data Dictionary

3.5.1 Entity Admin

Table 3.5.1: Data dictionary table for entity Admin

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK / FK	Reference Table
Admin ID	A001	VARCHAR (4)	AXXX			PK	
Admin Name	Ali	VARCHAR (50)			YES		
Admin Tel	0123456789 1	CHAR (11)	XXXXX XXXXX		YES		
Admin Email	Ali@gmail.com	VARCHAR (150)			YES		
Admin Password	Ali56789	VARCHAR (30)			YES		

3.5.2 Entity Staff

Table 3.5.2: Data dictionary table for entity Staff

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK/ FK	Reference Table
Staff ID	S001	VARCHAR (4)	SXXX			PK	
Staff Name	Jason	VARCHAR (50)			YES		
Staff Position	Counter	VARCHAR (50)		Front Desk Librarian, Technical Services Librarian	YES		
Staff Tel	01234567891	CHAR (11)	XXXXXXXXXX		YES		
Staff Email	Jason@gmail.com	VARCHAR (150)			YES		
Staff Password	Jason6789	VARCHAR (30)			YES		
AdminID						FK	Admin

3.5.3 Entity Student

Table 3.5.3: Data dictionary table for entity Student

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK / FK	Reference Table
Student ID	Stu001	VARCHAR (6)	Stu XXX			PK	
Student Name	Muthu	VARCHAR (50)			YES		
Student Course	AI	VARCHAR (50)			NO		
Student Tel	01234567891	CHAR (11)	XXXX XXXX XXX		YES		
Student Email	Muthu@student.utem.edu.my	VARCHAR (150)			YES		
StaffID						FK	Staff

3.5.4 Entity Book

Table 3.5.4: Data dictionary table for entity Book

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK /FK	Reference Table
BookID	B001	VARCHAR (4)	BXXX			PK	
Book Name	The Jungle book	VARCHAR (150)			YES		
Book Author	Siti	VARCHAR (50)			YES		
Book Publisher	Happy Shd	VARCHAR (150)			YES		
Book DatePublish	5/1/2025	DATE	DD/MM/YYYY		YES		
Book Category	Horror	VARCHAR (50)			NO		
Book Status	available	VARCHAR (10)		available, on loan	YES		

3.5.5 Entity Loan

Table 3.5.5: Data dictionary table for entity Loan

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK/FK	Reference Table
BorrowID	Bor0001	VARCHAR (7)	Bor XXXX			PK	
StaffID						PK, FK	Staff
BookID						PK, FK	Book
StudentID						PK, FK	Student
LoanStatus	Borrowing	VARCHAR (9)		Borrowing, Returned	YES		
DateOf Borrow	10/1/2025	DATE	DD/MM/YYYY		YES		
DueDate	10/2/2025	DATE	DD/MM/YYYY		NO		
DateOf Return	20/1/2025	DATE	DD/MM/YYYY		NO		
Fine	50.00	DOUBLE			NO		

3.5.6 Entity Review

Table 3.5.6: Data dictionary table for entity Review

Attribute Name	Content	Data Type	Format	Range	NOT NULL?	PK/FK	Reference Table
ReviewID	R0001	VARCHAR (5)	RXXXX			PK	
BookID						PK, FK	Book
StudentID						PK, FK	Student
Comment	Good, interesting story	VARCHAR (500)			NO		
StarRate	5	INT		1,2,3,4,5	YES		

3.6 Interface Design

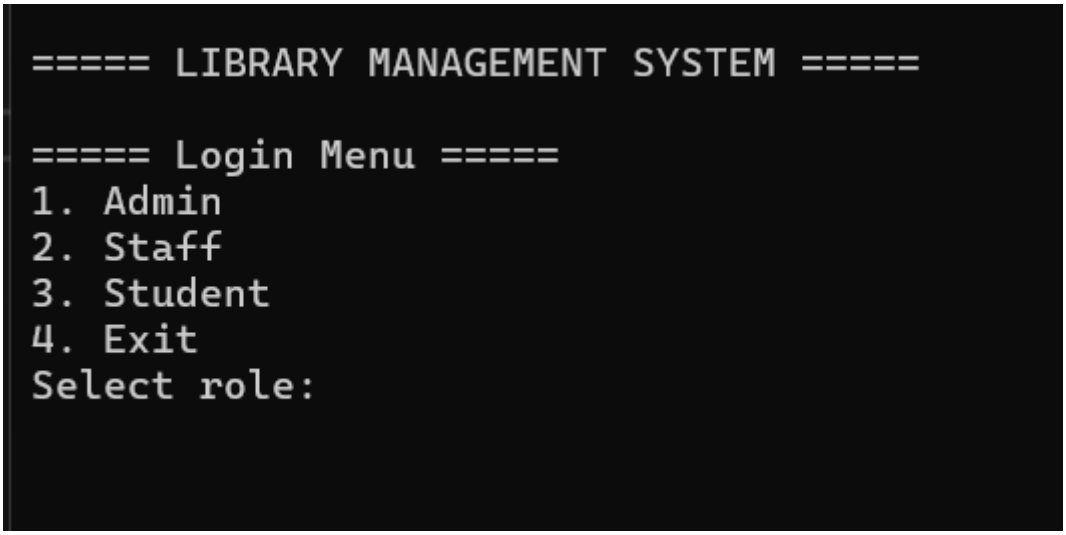

Figure 3.6.1: Login Menu

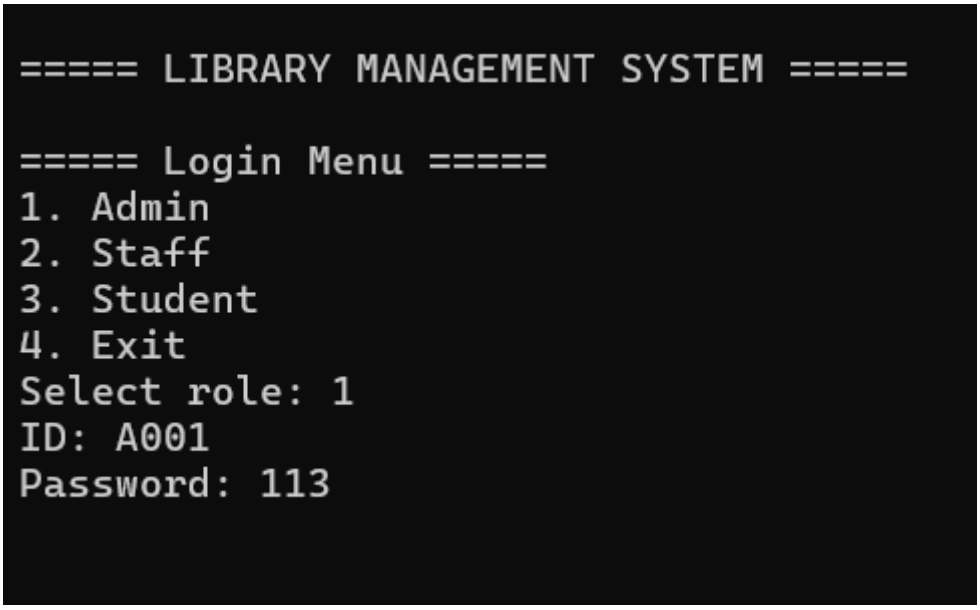

Figure 3.6.2: User Authentication

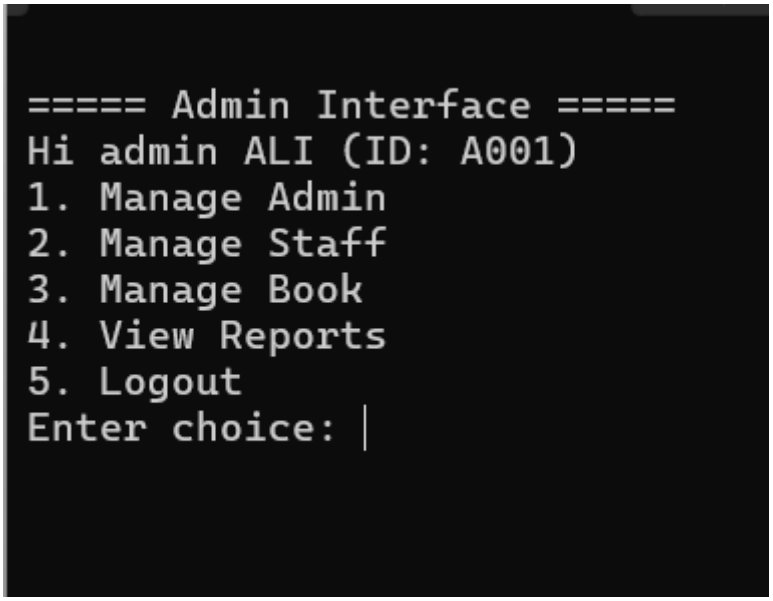

Figure 3.6.3: User Admin Menu

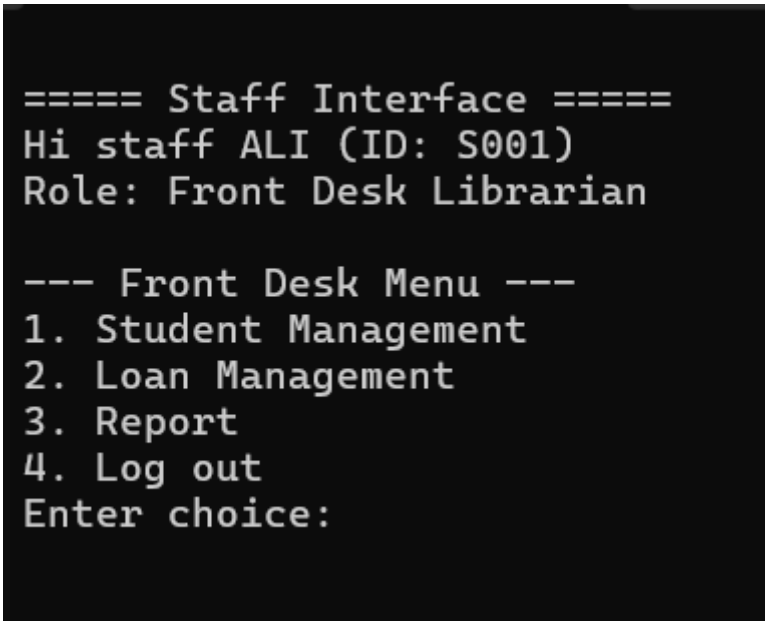

Figure 3.6.4: User Staff Menu

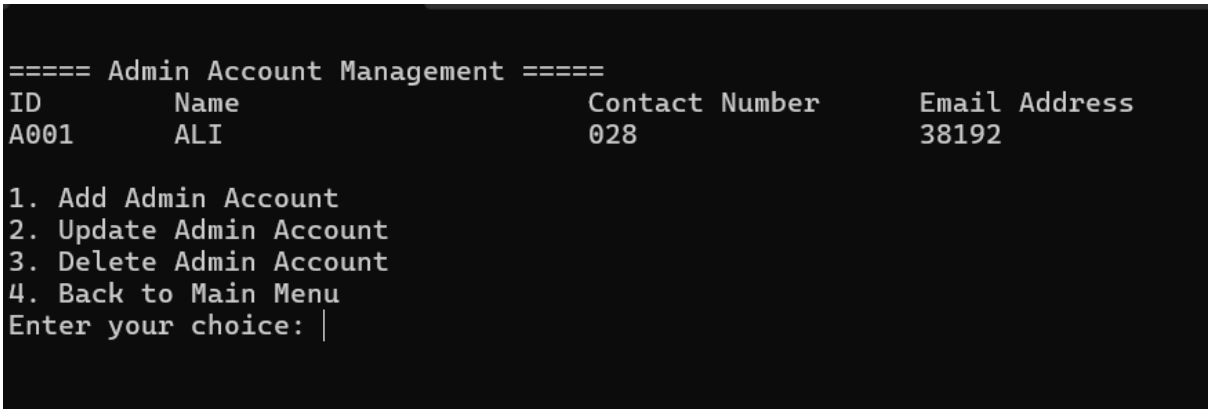

Figure 3.6.5: Entity data Management Interface (admin)

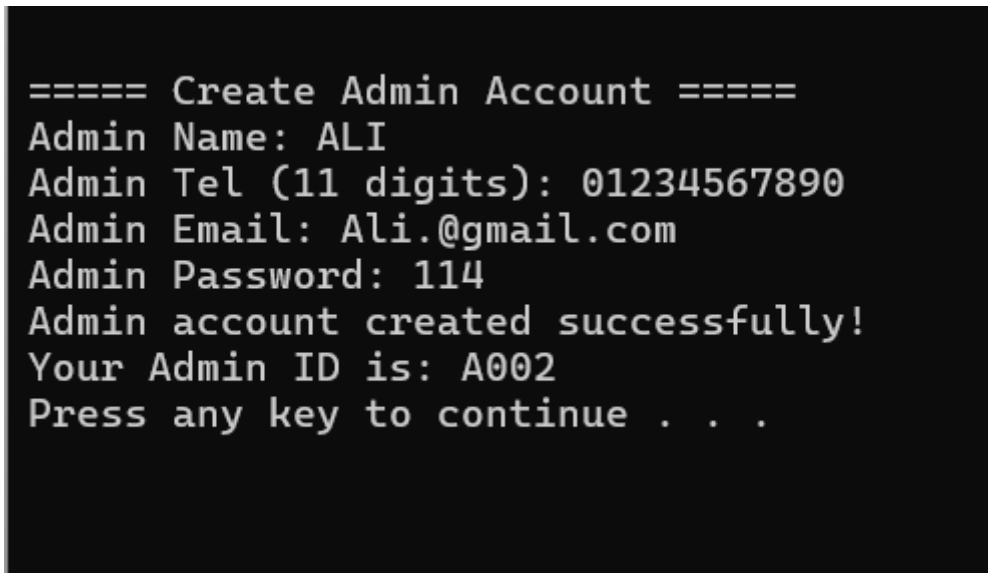

Figure 3.6.6: Add Interface

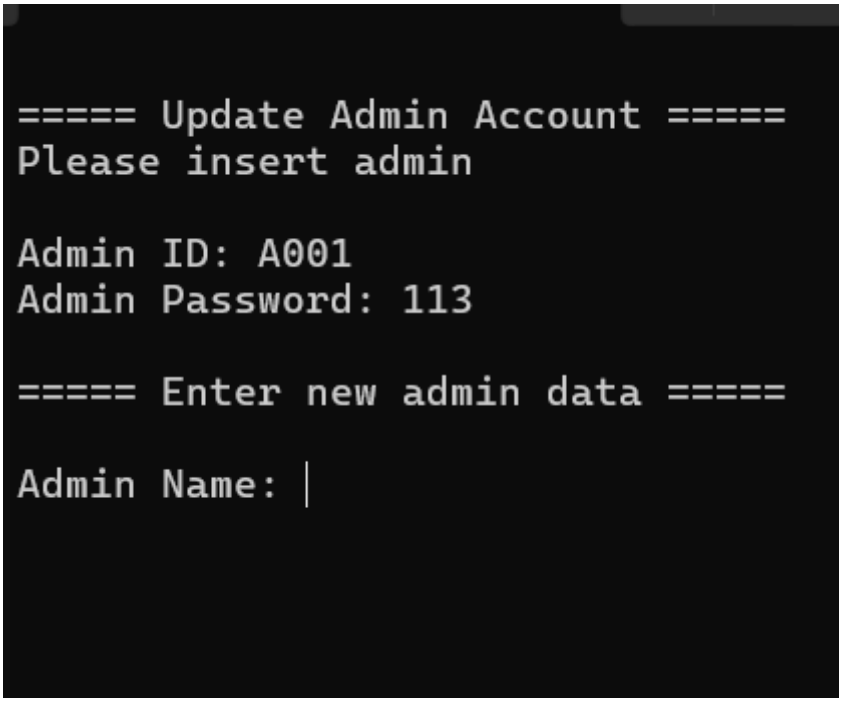

Figure 3.6.7: Update Interface

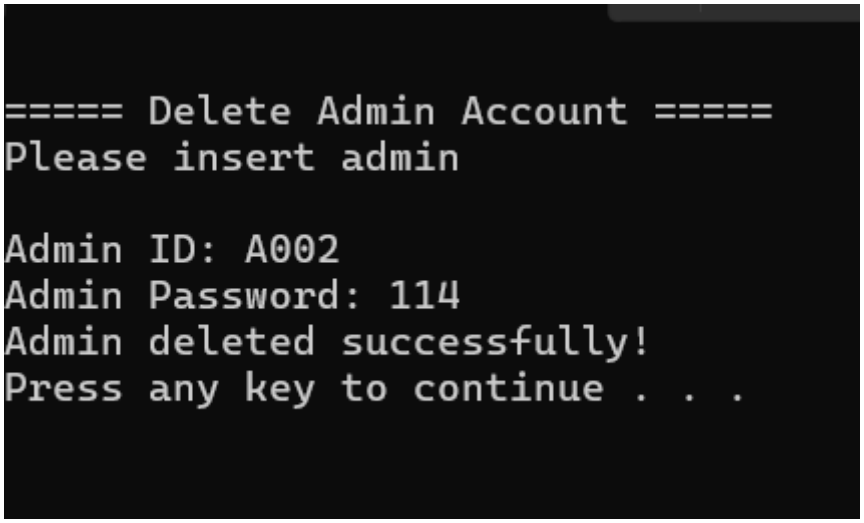

Figure 3.6.8: Delete Interface

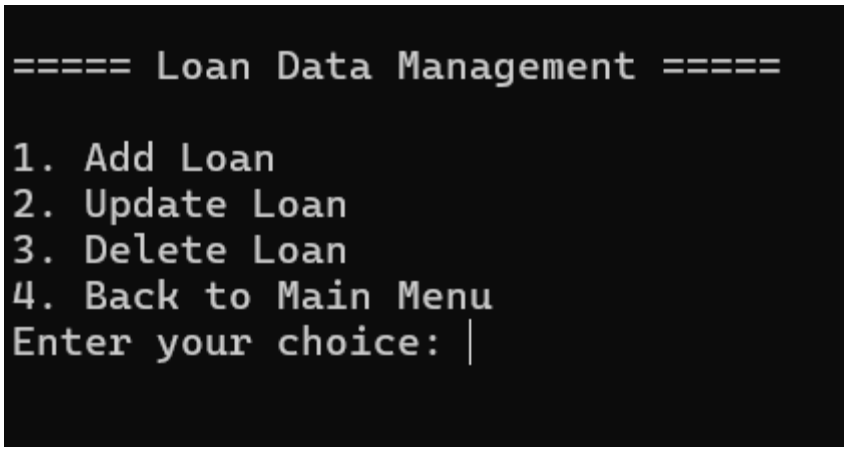

Figure 3.6.9: Entity data Management Interface (Loan)

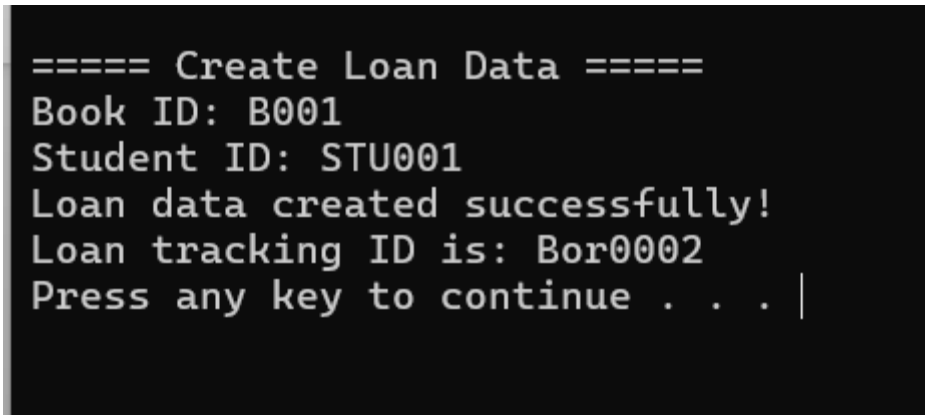

Figure 3.6.10: Add Loan Interface

```
===== Update Loan Data =====
Please insert loan

Loan ID: BOR0001

===== Loan Data =====
LoanID  BookID  StudentID   Status      Borrowed    Due         Returned    Fine
-------------------------------------------------------------------------------
Bor0001 B001    STU001      Returned    2026-01-10  2026-02-09  2026-01-10  0.00

Is this loan data
1. True
2. False
choice:1
1. Return Book
2. Fine payment
3. Exit
Choice: 2
Enter amount of fine: 0
Fine Payment Successful!
Press any key to continue . . .
```

Figure 3.6.11: Update Loan Interface

```
===== Delete Loan Data =====
Please insert loan

Loan ID: BOR0001

===== Loan Data =====
LoanID  BookID  StudentID   Status      Borrowed    Due         Returned    Fine
-------------------------------------------------------------------------------
Bor0001 B001    STU001      Returned    2026-01-10  2026-02-09  2026-01-10  0.00

Is this loan data
1. True
2. False
Choice: 1
Loan deleted Successful!
Press any key to continue . . .
```

Figure 3.6.12: Delete Loan Interface

```
Info: Student has already reviewed this book.
Do you want to overwrite the existing review? (y/n): Y
Enter Star Rating (1-5): 5
Enter Comment/Feedback: GOOD
Review updated successfully!
Press any key to continue . . .
```

Figure 3.6.13: Add Review Interface

```
===== Advanced Book Search =====
Enter Keyword (ID, Title, Author, Publisher, Date, Category): A

Search Results:
ID        Title                   Author              Publisher           Date           Status         Category
-------------------------------------------------------------------------------------------------------------------
B001      AKU                     A                   A                   2024-08-09     Available      HA
-------------------------------------------------------------------------------------------------------------------
Press any key to continue . . .
```

Figure 3.6.14: Book Searching

```
===== View Book Reviews =====
Enter Book ID to view reviews: B001

Reviews for: AKU
Student             Stars     Comment
-------------------------------------------------------------------------
ABU                 5         GOOD
-------------------------------------------------------------------------
Average Rating: 5.00 / 5.0
Press any key to continue . . .
```

Figure 3.6.15: Review Book

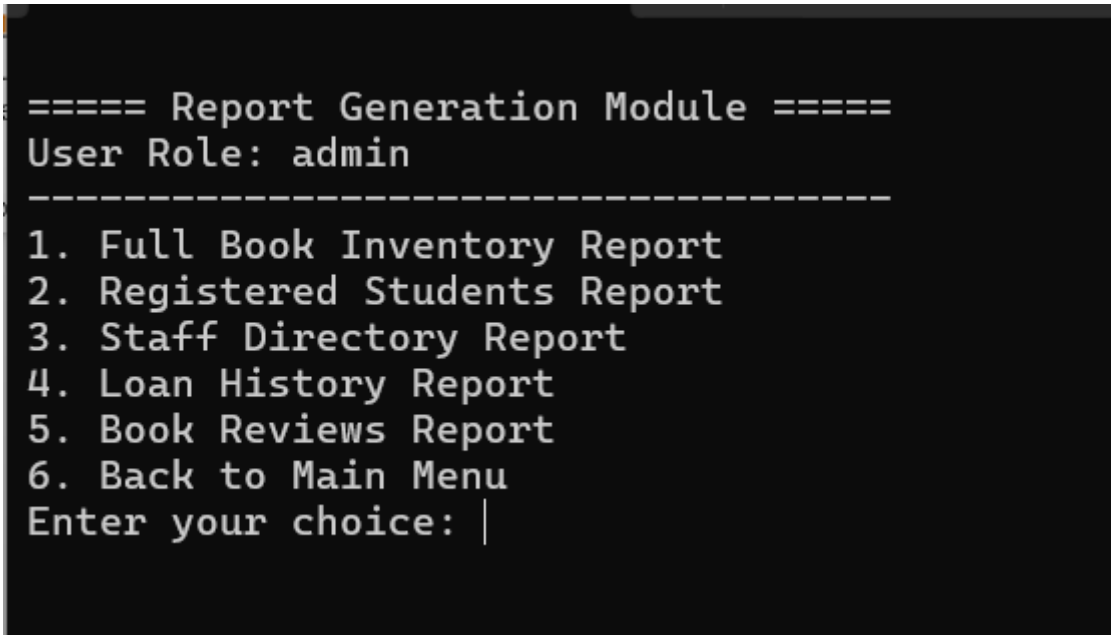

Figure 3.6.16: Report Selection

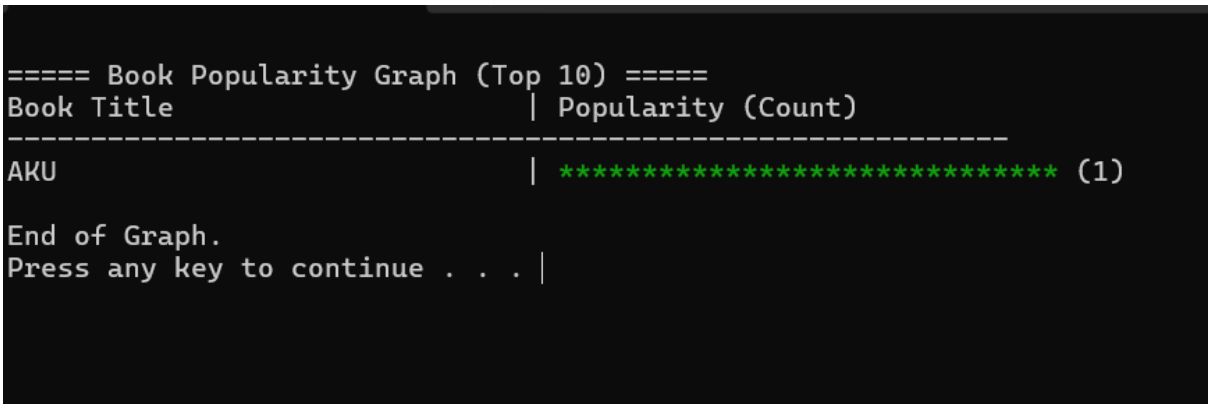

Figure 3.6.17: Book Popularity Graph

CHAPTER 4: IMPLEMENTATION

4.1 Naming Convention

```
class LoginModule {
private:
    const string server = "tcp://127.0.0.1:3306";
    const string username = "root";
    const string password = "";
    const string database = "librarymanagementdb";
    sql::Driver* driver;

public:
    LoginModule();
    void initDatabase();
    void checkAdminExists();
    void createAdmin();
    void loginMenu();
    bool verifyLogin(const string& role, const string& id,
const string& pass);
    bool checkDataNotExists(const string& role);
    void adminInterface(string id);
    void staffInterface(string id);
    void studentInterface();
};
```

Figure 4.1.1: Login Module function Naming

Figure 4.1.1 shows the LoginModule class, which manages all login-related operations in the library management system. It includes private members for database connection details (server, username, password, database) and a pointer to the MySQL driver (driver). The public methods handle tasks such as initializing the database (initDatabase()), checking and creating admin accounts (checkAdminExists() and createAdmin()), verifying login credentials (verifyLogin()), and directing users to role-specific interfaces (adminInterface(), staffInterface(), studentInterface()). The class follows standard naming conventions, with PascalCase for the class name, camelCase for methods and variables, and descriptive names for pointers and parameters, ensuring the code is clear, organized, and maintainable.

4.2 Function

```
void LoginModule::loginMenu() {
    int choice;
    do {
        string role, id, pass;
        cout << "\n===== Login Menu =====" << endl;
        cout << "1. Admin\n2. Staff\n3. Student\n4.
Exit\nSelect role: ";
        cin >> choice;

        // Determine role based on input
        switch (choice) {
        case 1: role = "admin"; break;
        case 2: role = "staff"; break;
        case 3: role = "student"; break;
        case 4: cout << "Exiting..."; return; // exit
        default:
            cout << "Invalid selection.\n";
            system("pause");
            system("cls");
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            continue;
        }
        if (role != "student") {
            if (checkDataNotExists(role)) {
                system("pause");
                system("cls");
                continue; // Skip if no data exists
            }
            cout << "ID: "; cin >> id;
            cout << "Password: "; cin >> pass;
            if (verifyLogin(role, id, pass)) {
                cout << "Login successful as " << role <<
"!\n";
                system("pause");
                if (role == "admin") {
                    system("cls");
                    adminInterface(id);
                }
                else if (role == "staff") {
                    system("cls");
                    staffInterface(id);
                }
            }
            else {
                cout << "Invalid ID or password.\n";
```

```
                system("pause");
                system("cls");
            }
        }
        else {
            system("cls");
            studentInterface();
        }
    } while (choice != 4); // loop until exit
}
```

Figure 4.2.1: Login Function Coding

Figure 4.2.1 illustrates the login menu function. This function prompts the user to choose whether to log in as an admin, staff, or student. It then checks the database to see if any admin or staff records exist. If they do, the user will be asked to enter their ID and password. When the verification function returns true, it indicates that the ID and password match, and the user can proceed to the respective interface. Since students do not have accounts and can only view books, they are directed straight to the student interface without any verification.

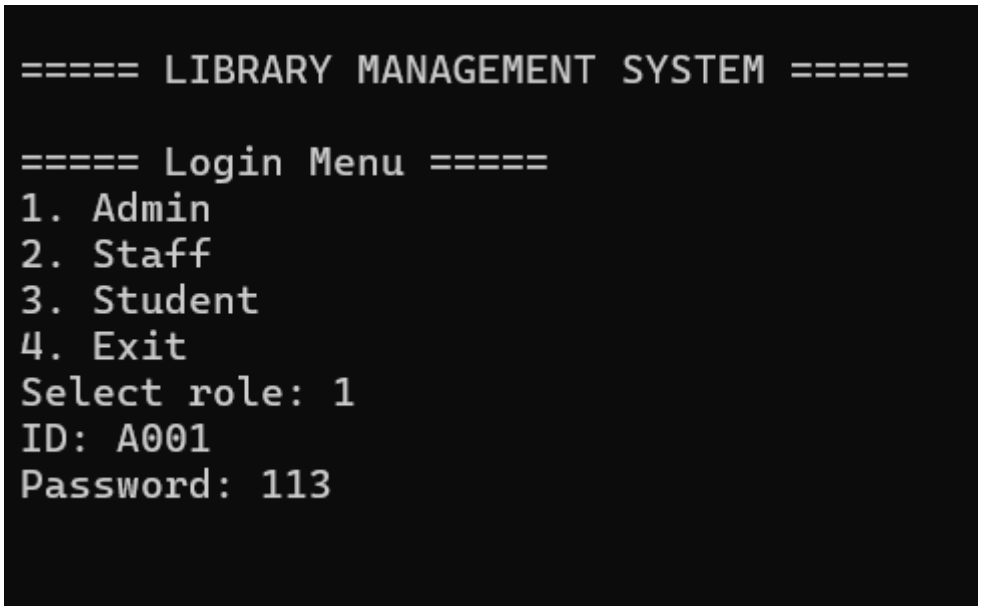

Figure 4.2.2: Login Function

4.3 Array

```
while (true) {
    cout << "Date of publish (YYYY-MM-DD): ";
    getline(cin, publishDate);

    // 1. Check length (must be 10 characters: "YYYY-MM-DD")
    if (publishDate.length() != 10) {
        cout << "Invalid format! Length must be 10 characters
(e.g., 2023-01-01).\n";
        continue;
    }

    // 2. Check delimiters (must have hyphens at index 4 and
7)
    if (publishDate[4] != '-' || publishDate[7] != '-') {
        cout << "Invalid format! Missing hyphens (Use YYYY-MM-
DD).\n";
        continue;
    }

    // 3. Check if all other characters are numbers
    bool isNumeric = true;
    for (int i = 0; i < 10; i++) {
        if (i == 4 || i == 7) continue; // Skip hyphens
        if (!isdigit(publishDate[i])) {
            isNumeric = false;
            break;
        }
    }

    if (!isNumeric) {
        cout << "Invalid format! Date must contain only
numbers and hyphens.\n";
        continue;
    }
    // 4. Logical Validation (Month 1-12, Day 1-31)
    int year = stoi(publishDate.substr(0, 4));
    int month = stoi(publishDate.substr(5, 2));
    int day = stoi(publishDate.substr(8, 2));

    if (month < 1 || month > 12) {
        cout << "Invalid month! Must be between 01 and 12.\n";
        continue;
    }
    if (day < 1 || day > 31) {
        cout << "Invalid day! Must be between 01 and 31.\n";
        continue;
    }
```

```
    // If we reach here, the date is valid
    break;
}
```

Figure 4.3.1: Array Example Code

Figure 4.4.1 shows the validation logic that treats the publishDate string essentially as a fixed-length character array of 10 elements. It relies on direct array indexing (using publishDate[i]) to verify the format: it specifically checks that the elements at index 4 and index 7 contain hyphens (-) to separate the date components, while a loop iterates through the remaining indices to ensure those array slots contain only numeric digits. Finally, distinct segments of this character array are extracted via substrings to convert the year, month, and day portions into integers, ensuring they fall within valid calendar ranges.

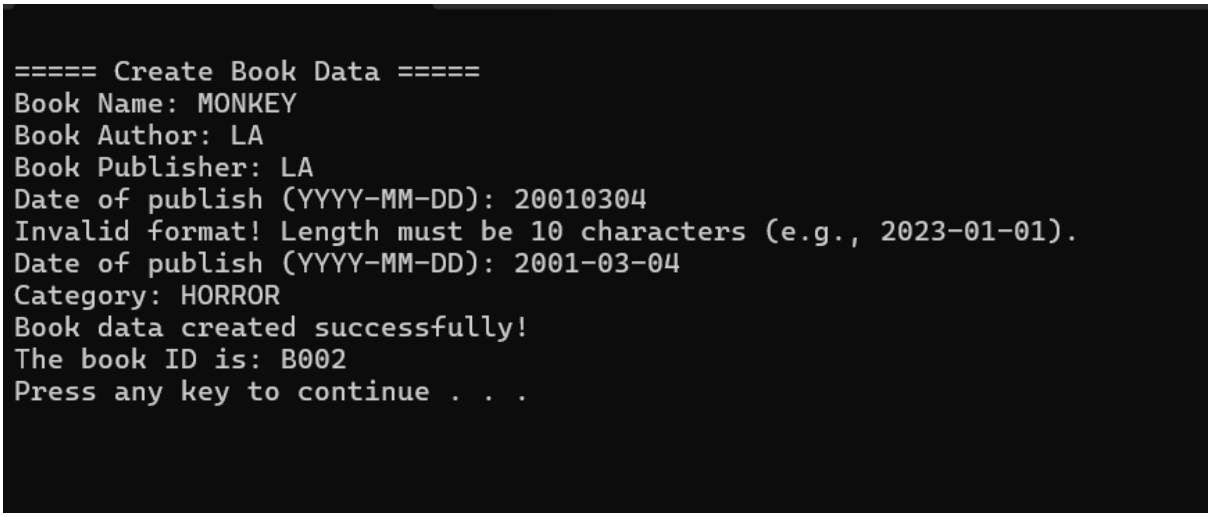

Figure 4.3.2: Array Example Output

4.4 Selection

```
void DataManagement::AdminManagement() {
    int choice;
    try {
        sql::Connection* con = driver->connect(server,
username, password);
        con->setSchema(database);
        sql::PreparedStatement* pstmt = nullptr;
        do {
            cout << "\n===== Admin Account Management =====" 
<< endl;
            cout << left
                << setw(10) << "ID"
                << setw(25) << "Name"
                << setw(20) << "Contact Number"
                << setw(30) << "Email Address"
                << "\n";
            pstmt = con->prepareStatement("SELECT* FROM
admin");
            sql::ResultSet* res = pstmt->executeQuery();
            while (res->next()) {
                cout << left
                    << setw(10) << res->getString("AdminID")
                    << setw(25) << shorten(res-
>getString("AdminName"), 25)
                    << setw(20) << res->getString("AdminTel")
                    << setw(30) << shorten(res-
>getString("AdminEmail"), 25)
                    << "\n";
            }
            cout << "\n1. Add Admin Account" << endl;
            cout << "2. Update Admin Account" << endl;
            cout << "3. Delete Admin Account" << endl;
            cout << "4. Back to Main Menu" << endl;
            cout << "Enter your choice: ";
            cin >> choice;
            switch (choice) {
            case 1:
                AdminAdd();    // call add function
                break;
            case 2:
                AdminUpdate();
                break;
            case 3:
                AdminDelete();
                break;
            case 4:
                cout << "Returning to main menu...\n";
```

```
                return;
            default:
                cout << "Invalid choice. Try again.\n";
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            }

        } while (choice != 4);
    }
    catch (sql::SQLException& e) {
        cerr << "Error checking " << e.what() << endl;
        return;
    }
```

Figure 4.4.1: Admin Management Coding

Figure 4.4.1 shows the function that manages admin accounts by first displaying all existing admins in a formatted table. It then presents a menu where the user must select an action: add, update, or delete an admin account. Based on the user's selection, the function calls the corresponding action. Choosing option 4 returns the user to the main menu. Any database errors that occur are properly handled and displayed.

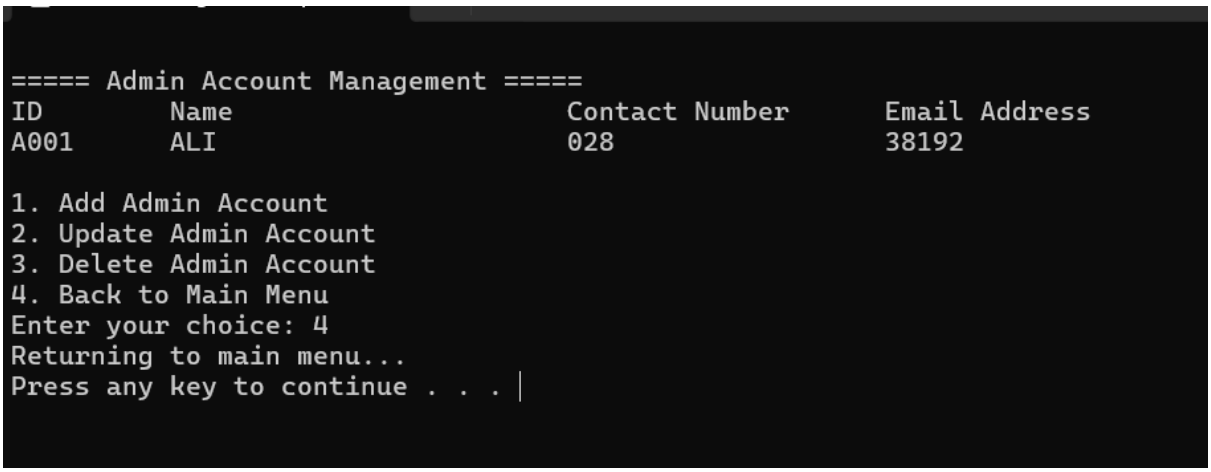

Figure 4.4.2: Admin Management Output

4.5 Control

```
bool LoginModule::verifyLogin(const string& role, const
string& id, const string& pass) {
    string table, idField, passField;
    if (role == "admin") {
        table = "admin";
        idField = "AdminID";
        passField = "AdminPassword";
    }
    else if (role == "staff") {
        table = "staff";
        idField = "StaffID";
        passField = "StaffPassword";
    }
    else {
        cerr << "Invalid role specified.\n";
        return false;
    }

    try {
        sql::Connection* con = driver->connect(server,
username, password);
        con->setSchema(database);

        string query = "SELECT * FROM " + table + " WHERE " +
idField + "=? AND " + passField + "=?";
        sql::PreparedStatement* pstmt = con-
>prepareStatement(query);
        pstmt->setString(1, id);
        pstmt->setString(2, pass);

        sql::ResultSet* res = pstmt->executeQuery();
        bool found = res->next();

        delete res;
        delete pstmt;
        delete con;

        return found;
    }
    catch (sql::SQLException& e) {
        cerr << "Login check failed: " << e.what() << endl;
        return false;
    }
}
```

Figure 4.5.1: Verify Login Coding

Figure 4.5.1 shows the function that checks a user's login using control flow to determine the appropriate actions based on the selected role. It first uses an if–else structure to choose the correct database table and fields for either admin or staff. Then, it connects to the database and executes a query to verify whether the entered ID and password match any record. Afterwards, another control decision checks the result and returns true if a match is found, or false otherwise. If any error occurs, the catch block handles it and returns false.

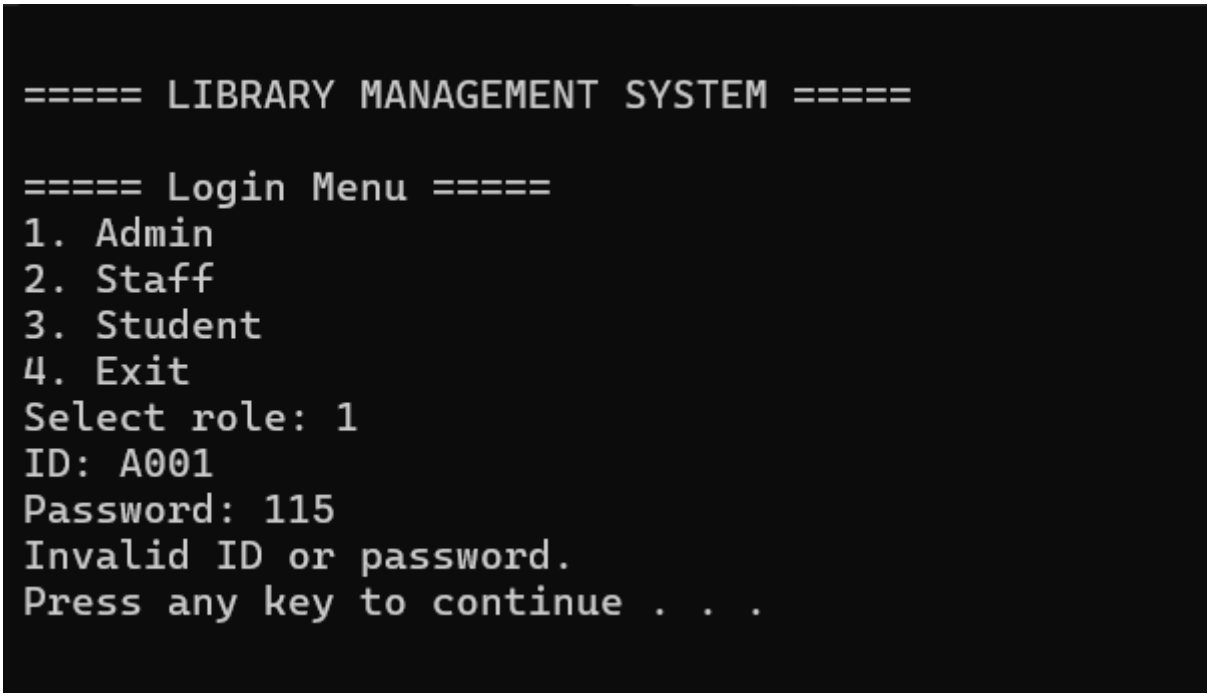

Figure 4.5.2: Verify Login Output

4.6 Pointer

```
void LoginModule::initDatabase() {
    try {
        sql::Connection* con = driver->connect(server,
username, password);
        con->setSchema(database);
        cout << "Database connected successfully.\n";
        delete con;
    }
    catch (sql::SQLException& e) {
        cout << "Database connection failed: " << e.what() <<
endl;
        exit(1);
    }
}
```

Figure 4.6.1: Initialise Database

Figure 4.6.1 shows the initDatabase() function, which establishes a connection to the database using a pointer. It creates a dynamically allocated Connection object with sql::Connection* con, allowing the program to access the database through this pointer. The function then sets the database schema using con->setSchema(database) and prints a success message if the connection is successful. Afterward, the pointer is deleted with delete con to free the allocated memory and prevent leaks. If any error occurs during the connection, the catch block handles it by printing an error message and exiting the program.

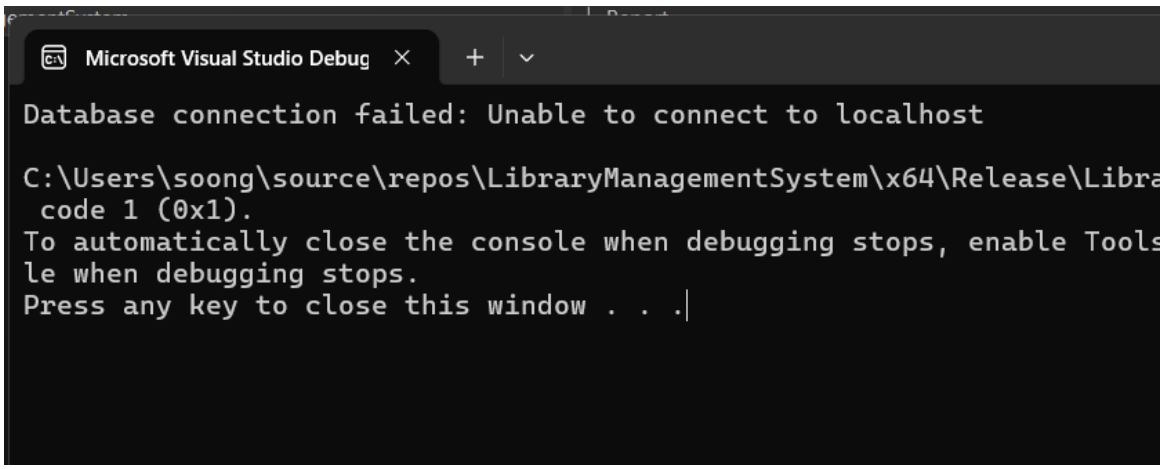

Figure 4.6.2: Initialise Database Output

4.7 Error Handling

```
cout << "\n===== Staff Interface =====\n";
cout << "Hi staff " << name << " (ID: " << id << ")\n";
cout << "Role: " << position << "\n";
cout << "\n--- Front Desk Menu ---\n";
cout << "1. Student Management\n";
cout << "2. Loan Management\n";
cout << "3. Report\n";
cout << "4. Log out\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice) {
case 1:
    system("cls");
    staff.StudentManagement(id);
    break;
case 2:
    system("cls");
    Booking.LoanManagement(id);
    break;
case 3:
    system("cls");

    reportModule.ReportMenu("staff");
    break;
case 4:
    cout << "Logging out...\n";
    system("pause");
    system("cls");
    break;
default:
    cout << "Invalid choice.\n";
    system("pause");
    system("cls");
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
```

Figure 4.7.1: Staff Selection Coding

The code in Figure 4.7.1 implements a robust menu system using a do-while loop that continuously prompts the user until the exit condition (option 4) is met, ensuring the program doesn't terminate prematurely. Inside the loop, a switch statement directs the flow based on the user's integer input, while the default case acts as a critical error handler for both invalid numbers and data type mismatches (such as entering letters instead of digits). To prevent the program from entering an infinite loop when non-integer input causes cin to fail, the code executes cin.clear() to reset the input stream's error flags and cin.ignore() to flush the invalid characters from the memory buffer, allowing the system to recover cleanly and request a new input.

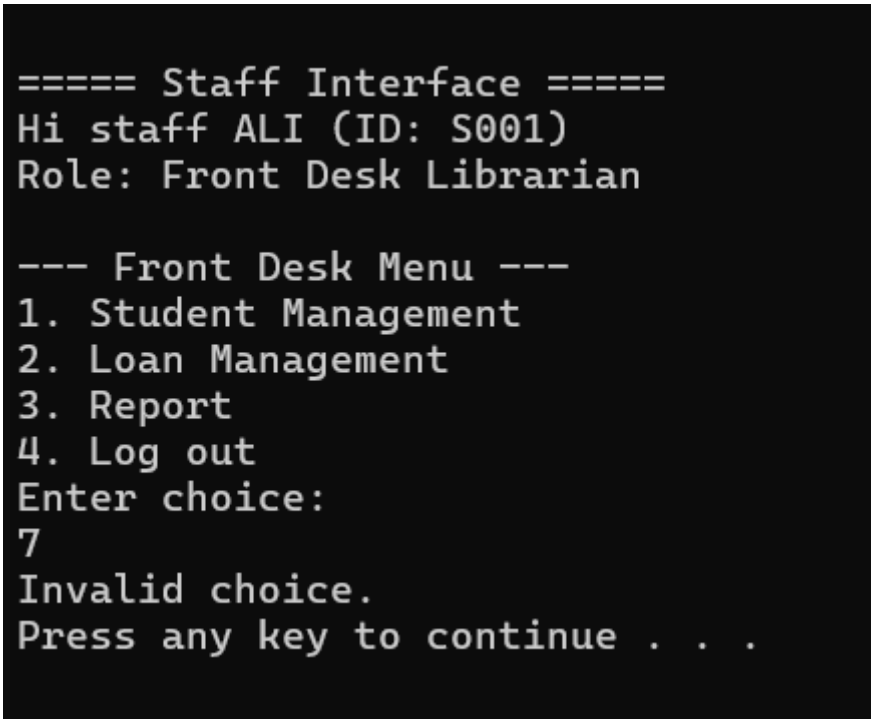

Figure 4.7.2: Staff Selection Output

4.8 Business Rule Implementation

```
sql::ResultSet* res = pstmt->executeQuery();
res->next();

int count = res->getInt("count");

bool notExists = (count == 0);

if (notExists) {
    cout << "No " << role << " found in database.\n";
}
```

Figure 4.8.1: Check Any Existence of Data Coding

Figure 4.8.1 shows a section of code that demonstrates Business Rule Implementation in C++. This code checks whether any records exist for a specific role (admin, staff, or student) in the database, enforcing a rule that certain operations require at least one existing record. First, it executes a SQL query using a pointer to a ResultSet (res) and moves to the first row with res->next(). It then retrieves the count of records with res->getInt("count"). The boolean variable notExists is set to true if the count is zero, indicating no records exist. Finally, an if statement enforces the business rule: if no records are found, it outputs a message like "No admin found in database.", ensuring the program does not proceed with operations that require existing data. This is a clear example of business rules being implemented in code to maintain data integrity and control program flow.

```
===== Student Data Management =====
ID        Name                    Course         Contact Number      Email Address
No student found in database.

1. Add Student Account
2. Update Student Account
3. Delete Student Account
4. Back to Main Menu
Enter your choice:
```

Figure 4.8.2: Check Any Existence Output

4.9 Complex Calculation Implementation

```
// 4. CALCULATE AND UPDATE FINE
pstmt = con->prepareStatement(
    "UPDATE loan SET FINE = GREATEST(0, DATEDIFF(DateOfReturn,
DueDate) * 5) WHERE BorrowID = ?"
);
pstmt->setString(1, LoanID);
pstmt->execute();
delete pstmt;
```

Figure 4.9.1: Fine Calculation Coding

This code snippet executes a direct SQL update to mathematically determine and persist the overdue penalty by calculating the temporal variance between the actual DateOfReturn and the scheduled DueDate using the DATEDIFF function. This day-count delta is multiplied by a coefficient of 5—representing the daily fine rate—before being processed by the GREATEST(0, ...) function, which effectively clamps the result to a lower bound of zero; this ensures that an early return, which would mathematically yield a negative product (e.g., -2 days × 5 = -10), is logically corrected to zero to prevent the system from assigning a negative debt, while strictly positive values are committed to the database as the final FINE for the specific BorrowID.

```
--- Loan & Transaction Report ---
LoanID  BookID  StudentID   Status      Borrowed    Due         Returned    Fine
-------------------------------------------------------------------------------
Bor0001 B001    Stu001      Returned    2025-01-12  2025-02-11  2026-01-12  1675.00
Bor0002 B002    Stu002      Borrowing   2026-01-12  2026-02-11  00-00-00    0.00
```

Figure 4.9.2

Figure 4.9.2: Fine Calculation Output

4.10 Report Generation for Analysis Implementation

```
void Report::generateTop5View() {
    system("cls");
    cout << "\n--- Top 5 Most Active Students ---\n";
    try {
        sql::Connection* con = driver->connect(server,
username, password);
        con->setSchema(database);

        // Query: Join tables to count loans per student,
ordered descending
        sql::PreparedStatement* pstmt = con->prepareStatement(
            "SELECT s.StudentID, s.StudentName,
s.StudentCourse, COUNT(l.BorrowID) AS TotalBorrows "
            "FROM student s "
            "JOIN loan l ON s.StudentID = l.StudentID "
            "GROUP BY s.StudentID "
            "ORDER BY TotalBorrows DESC "
            "LIMIT 5"
        );
        sql::ResultSet* res = pstmt->executeQuery();

        cout << left
            << setw(6) << "Rank"
            << setw(10) << "ID"
            << setw(25) << "Name"
            << setw(15) << "Course"
            << setw(15) << "Total Borrows" << endl;
        cout << "-----------------------------------------
----------------------\n";

        int rank = 1;
        bool hasData = false;
        while (res->next()) {
            hasData = true;
            cout << left
                << setw(6) << rank++
                << setw(10) << res->getString("StudentID")
                << setw(25) << shorten(res-
>getString("StudentName"), 23)
                << setw(15) << res->getString("StudentCourse")
                << setw(15) << res->getInt("TotalBorrows")
                << endl;
        }

        if (!hasData) {
            cout << "No borrowing history found.\n";
        }
```

```
        delete res; delete pstmt; delete con;
        cout << "\nEnd of Report.\n";
        system("pause");
        system("cls");
    }
    catch (sql::SQLException& e) {
        cerr << "Report Error: " << e.what() << endl;
        system("pause");
        system("cls");
    }
}
```

Figure 4.10.1: Generate Top 5 Student Report Coding

This function implements report generation for analysis by leveraging a relational database query to transform raw transaction data into actionable insights regarding student engagement. It executes an SQL command that joins the student and loan tables, utilizing the COUNT aggregate function and GROUP BY clause to calculate the total number of loans per student, while the ORDER BY ... DESC and LIMIT 5 clauses isolate the most active users. The resulting analytical data is then retrieved and formatted into a structured table using C++ stream manipulators like setw, providing a clear, ranked visualization of the top borrowers to support administrative monitoring and decision-making.

```
--- Top 5 Most Active Students ---
Rank  ID        Name                          Course          Total Borrows
-------------------------------------------------------------------------
1     Stu001    AHMAD                         FTMK            3
2     Stu005    AJ                            AJ              1
3     Stu004    ff                            ff              1
4     Stu003    kk                            kk              1
5     Stu002    SITI                          FTKEK           1

End of Report.
Press any key to continue . . .
```

Figure 4.10.2: Top 5 Student Report Output

CHAPTER 5: CONCLUSION

5.1 Constraints

a) **Database Connectivity Issues:**

The system frequently experienced failures when attempting to connect to the MySQL server, which directly prevented CRUD operations from executing.

b) **Trigger and Logic Anomalies:**

Incorrect MySQL trigger settings caused unexpected behaviours, such as data corruption and "Duplicate Entry" errors when generating auto-incremented primary keys (e.g., Student IDs).

c) **Complex Code Structure:**

Looping Logic:

The use of complex, nested do-while loops for menus and error handling made debugging and maintaining the control flow difficult.

High Coupling:

The code was highly coupled, meaning that modifying a single function often caused unintended bugs in related modules (regression risks), requiring extensive re-testing.

d) **SQL Implementation Challenges:**

Syntax Errors:

Combinations of connectivity issues and syntax errors in Prepared Statements made the system functionally unreliable at times.

Complex Joins:

Writing SQL queries that required joining multiple tables (e.g., linking reviews to student names) was difficult to implement.

e) **User Interface Inconsistencies:**

It was challenging to consistently implement UI refreshes (using system("cls")) across all sub-menus and error states, leading to a sometimes-cluttered interface.

f) **Design Decisions:**

Significant time was required to determine the optimal balance of information for reports, such as deciding whether to display full timestamps or just dates to maintain table alignment.

5.2 Future Improvements

a) Automated Email Notification System

Currently, the system relies on students manually checking their due dates or staff verbally reminding them. To improve borrowing compliance and reduce overdue rates, the system should integrate an **SMTP (Simple Mail Transfer Protocol)** library. This enhancement would allow the application to automatically send transactional emails to the addresses stored in the student table. Key features would include:

Due Date Reminders: Automatically sending a "Courtesy Notice" 3 days before a book is due.

Overdue Alerts: Instantly triggering an "Overdue Notice" with the calculated fine amount once the return deadline has passed.

Transaction Receipts: Sending digital confirmation emails whenever a book is borrowed or returned, creating a reliable paper trail for students.

b) **Password Security and Role-Based Access Control (RBAC)** The current implementation stores user passwords in plain text, which presents a significant security vulnerability. Future iterations of the system must implement cryptographic security measures to protect user data.

Password Hashing: Instead of storing raw passwords, the system should utilize secure hashing algorithms (such as **SHA-256** or **bcrypt**) to encrypt credentials before saving them to the database. This ensures that even if the database is compromised, user passwords remain secure.

Strict RBAC Enforcement: While the system currently separates menus by role, the security layer should be deepened to prevent "privilege escalation." This involves implementing session tokens or strict server-side checks to ensure that a student cannot access staff-level functions (like LoanUpdate) simply by manipulating the application code or memory.

CHAPTER 6: REFERENCES

Integrated Library Management System，- S. A. V.- R. S. G. S. International Journal for Multidisciplinary Research (2024)

JK Bengkel 1, “Buku Panduan Bengkel 1 BITU 2913”, FTMK (2025)

Bjarne Stroustrup, “Programming: Principles and Practice Using C++ (C++ In-depth)”, Addison-Wesley Professional (2024)

McLaughlin, M., “"Oracle Database 12c PL/SQL Programming"”, Oracle Press / McGraw-Hill Education (2023)

Sveta Smirnova, Alkin Tezuysal, "MySQL Cookbook: Solutions for Database Developers and Administrators (4th Edition)", O'Reilly Media, 2022.

David Thomas, Andrew Hunt, " The Pragmatic Programmer: Your Journey to Mastery (20th Anniversary Edition) " Addison-Wesley Professional, 2019.